МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования

САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ

СИТЕМЫ БАЗ ДАННЫХ

(Часть 1)

Учебное пособие

ГУАП

Санкт-Петербург

2021

УДК

ББК

Рецензенты:

кандидат технических наук, доцент *Т. Н. Костюнина*;

кандидат технических наук, доцент *С. В. Щекин*

Утверждено

редакционно-издательским советом университета

в качестве учебного пособия

Протокол №___ от ___ _____ 2021 г.

Богословская Н. В.

Системы баз данных (Часть 1): учебное пособие / Н. В. Богословская, А. В. Бржезовский. – СПБ: ГУАП 2021. – 83 с.

В учебное пособие включены описания основных конструкций языка SQL (Structured Query Language), обеспечивающих взаимодействие с базами данных: создание и ведение баз данных, формирование запросов. Рассмотрены управляющие конструкции SQL, возможности по обработке данных посредством написания хранимых процедур и обеспечения активной целостности с помощью триггеров.

Примеры синтаксических конструкций, представленные в пособии, реализованы на диалекте Transact-SQL для Microsoft SQL Server 20012..2019, они могут иметь незначительные отличия от других диалектов языка и для других версий Microsoft SQL Server.

Учебное пособие предназначено для студентов, обучающихся по направлению 09.03.02 — Информационные системы и технологии.

УДК

ББК

1. СОЗДАНИЕ ТАБЛИЦ БАЗЫ ДАННЫХ

1.1 Базы данных и СУБД

Реляционная база данных (БД) или Data Base (DB) – это совокупность связанных таблиц для хранения информации об объектах (процессах, явлениях, фактах) некоторой предметной области. Для разработки баз данных и программных приложений, работающих с базами данных, часто используется язык SQL (Structured Query Language). Интерпретацию инструкций, формулируемых на языке SQL, хранение данных, обеспечение многопользовательской работы с данными, защиту, резервное копирование, восстановление данных и реализацию многих других функций обеспечивает специальный класс программных продуктов, получивший название систем управления базами данных (СУБД) или Data Base Management System (DBMS).

В настоящее время в связи с активным использованием систем, построенных в архитектуре клиент-сервер, в качестве синонима термина СУБД, часто используется термин SQL-сервер. Примерами таких продуктов могут служить Oracle Database Server, SAP Adaptive Server Enterprise (ASE), SAP SQL Anywhere (SQLA), Microsoft SQL Server (MS SQL), IBM DB2 Data Server (DB2) и др. Данные продукты относятся к классу RDBMS (Relational DBMS) или ORDBMS (Object-Relational DBMS). Как правило, в состав любой СУБД или SQL-сервера входит приложение для администрирования баз данных и приложение, обеспечивающее выполнение запросов к БД, в случае MS SQL эти возможности совмещены в среде SQL Server Management Studio, далее по тексу – Management Studio или SSMS. Для работы с БД данное приложение должно быть запущено (***Пуск\Все программы\Microsoft SQL Server …\Среда SQL Server Management Studio***) и соединены (кнопка или команда ***Соединить/Connect***) с SQL-сервером (***Компонент Database Engine\(local)\Проверка подлинности Windows***).

Создание БД осуществляет следующий оператор языка SQL:

```
create database <имя БД>[1]
go
```

Кнопкой или командой ***Создать запрос/New Query*** откройте специальное окно для выполнения операторов языка SQL (окно запросов) и введите, например, следующие операторы:

```
use master
go
create database Университет
go
use Университет
go
```

Выполнение операторов (кнопка или команда ***! Выполнить/! Extcute***) приведет к созданию базы данных. В данном случае имя БД – ***Университет*** (имя БД должно быть уникально в рамках сервера), оператор ***use*** задает БД, используемую по умолчанию, ***master*** – имя системной БД, создаваемой автоматически при установке MS SQL. Созданная база данных появляется в дереве объектов (окно ***Обозреватель объектов/Object Explorer***) после обновления узла ***Базы данных/Databases*** (команда контекстного меню ***Обновить/Refresh***). Удаление БД реализует оператор ***drop database***, имеющий аналогичный синтаксис.

Альтернативный способ создания БД — команда ***Создать базу данных…/NewDatabase…***, вызываемая из контекстного меню узла ***Базы данных/Databases*** дерева объектов, альтернативный способ удаления БД — команда ***Удалить/Delete***, вызываемая из контекстного меню узла, соответствующего удаляемой БД в дереве объектов.

---

[1] Скобки < > — указывают на необходимость подстановки синтаксической конструкции, в данном случае — идентификатора.

1.2 Таблицы

Для сохранения информации в БД необходимо создать одну или несколько таблиц. Для создания таблицы нужно выполнить оператор языка SQL ***create table***, его упрошенный синтаксис приведен ниже:

```
create table <имя таблицы> (
   <имя столбца 1> <тип 1>
   [ , <имя столбца 2> <тип 2>
   [ , … ] ]1 )
go
```

Альтернативный способ создания таблицы — команда ***Создать таблицу…/New Table…***, вызываемая из контекстного меню узла ***Таблицы/Tables*** дерева объектов, подчиненного узлу, соответствующему БД, в которой создается таблица. Созданная таблица появляется в дереве объектов (окно ***Обозреватель объектов/Object Explorer***) после обновления узла ***Таблицы/Tables*** (команда контекстного меню ***Обновить/Refresh***).

Основные типы данных языка SQL приведены в таблице 1.1. [Типы данных (Transact-SQL) - SQL Server | Microsoft Docs]

Таблица 1.1.

Типы данных MS SQL

Тип	Форма записи	Размерность	Пример
char[(n)] или character[(n)]	Последовательность символов длины ***n*** (при однобайтовой кодировке), заключенных в знаки апострофа, если ***n*** опущено – 1 символ; для хранения данных всегда отводится поле в ***n*** байт.	1 .. 8000	char (5) ‘abcd’
varchar[(n)]	См. ***char[(n)]*** для хранения данных отводится поле	1 .. 8000	varchar (50)

[1] Скобки [ ] — указывают, что синтаксическая конструкция может быть опущена, т. е. таблица может состоять из одного или нескольких столбцов, < > — указывают на необходимость подстановки синтаксической конструкции, в данном случае идентификатора таблицы, столбца, типа данных.

	длины, соответствующей числу фактически заданных символов.		
text[1] или varchar(max)	Последовательность символов переменной длины большой размерности.	2^31–1 (2 ГБ)	text varchar(max)
nchar[(n)]	То же, что и ***char*** в кодировке Unicode; ***n*** задает число символов (при двухбайтной кодировке).	1 .. 4000	nchar(1)
nvarchar[(n)]	То же, что и ***varchar*** в кодировке Unicode.	1 .. 4000	nvarchar (20)
ntext[2] или nvarchar(max)	То же, что и ***text*** в кодировке Unicode.	2^30–1 (1 ГБ)	ntext nvarchar(max)
bit	0 или 1.	0 .. 1	0
tinyint	Целое без знака.	0 .. 255	11
smallint	Целое со знаком.	–2^15 .. 2^15–1	–32233
int	Целое со знаком.	–2^31 .. 2^31–1	2111222333
bigint	Целое со знаком.	–2^63 .. 2^63–1	2111222333444
decimal[(p[,s])] или numeric[(p[,s])]	Вещественное число с фиксированной точкой (всего ***p*** значащих цифр, ***s*** значащих цифр после «.», по умолчанию ***p*** равно 18, ***s*** равно 0).	–10^38+1.. 10^38-1	–12.44
real	Вещественное число с плавающей точкой, для хранения отводится 4 байта, эквивалентно ***float(24)***.	–3,40E+38.. –1,18E–38,0 и 1,18E–38..	3.3E–5

[1] Сохранен для совместимости с предыдущими версиями, не рекомендован к использованию.
[2] Сохранен для совместимости с предыдущими версиями, не рекомендован к использованию.

		3,40E+38	
float[(n)]	Вещественное число с плавающей точкой, для хранения отводится 4 байта (7 цифр), если ***n*** в диапазоне 1..24, 8 байт (15 цифр), если ***n*** – 25..53.	–1,79E+308.. –2,23E–308,0 и 2,23E–308.. –1,79E+308	–3.3E5
smallmoney	Вещественное число с фиксированной точкой с 4-мя знаками после «.».	–214748,3648 ..214748,3647	–55.3333
money	Вещественное число с фиксированной точкой с 4-мя знаками после «.».	примерно –900..+900 триллионов	33.4455
smalldatetime[1]	Дата и время с точностью до секунд, дата от времени отделяется пробелом, разделителями в поле даты служат «–» или «/»,разделителем в поле времени – «:».	01/01/1900.. 06/06/2079	'2016-05-08 12:35:29'
datetime[2]	Дата и время с тремя знаками для долей секунд, значения долей секунд от времени отделяются символом «.».	01/01/1753 .. 31/12/9999	'2016-05-08 12:35:29.999'
date	Дата.	01/01/0001 .. 31/12/9999	'2016-05-08 12:35:29.999'
time[(n)]	Время с ***n*** знаками для долей секунд, ***n*** должно быть в диапазоне 1 .. 7, по умолчанию 7.	00:00:00 .. 23:59:59.9999999	'10:10:10.9999999'
datetime2[(n)]	Комбинация ***date*** и ***time[(n)]***.	—	—
datetimeoffset[(n)]	Комбинация **date** и ***time[(n)]*** с указанием смещения часового пояса.	—	'2016/05/08 12:35:29.1234567 +12:15'

[1] Не рекомендован к использованию в новых проектах.
[2] Не рекомендован к использованию в новых проектах.

binary[(n)]	Двоичные данные фиксированной длины, если ***n*** опущено – 1 байт.	1 .. 8000	—
varbinary[(n)]	Двоичные данные переменной длины, если ***n*** опущено – 1 байт.	1 .. 8000	—
image[1] или varbinary(max)	Двоичные данные переменной длины.	2^31–1 (2 ГБ)	—
timestamp[2] или rowversion	Автоматически формируемый при вставке и модификации код версии строки.	—	—
uniqueidentifier	Глобальный уникальный идентификатор (GUID).	—	—
xml	Синтаксически правильный XML-документ.	2^31–1 (2 ГБ)	—
hierarchyid	Специальный тип данных для хранения информации, представляемой в форме дерева.	—	'/2/1/2/'

Ниже приведен пример создания таблицы ***Студент*** (***Ном_зач*** – номер зачетной книжки):

```
create table Студент (
   Ном_Зач int,
   ФИО varchar(50),
   Группа char(5) )
go
```

Для удаления таблицы служит оператор ***drop table <имя таблицы>***. Альтернативный способ удаления таблицы — команда ***Удалить/Delete***, вызываемая

[1] Сохранен для совместимости с предыдущими версиями, не рекомендован к использованию.
[2] Отличается от типа данных ***timestamp***, определенного в стандарте ISO. В последних версиях MS SQL заменен на ***rowversion***.

из контекстного меню узла, соответствующего удаляемой таблице в дереве объектов.

В определении столбцов таблицы можно помимо задания типа данных указывать директивы:

(i) ***null*** или ***not null*** — разрешение/запрет NULL-значений (NULL – специальная константа языка SQL, показывающая, что при вставке данных в таблицу не было задано значение соответствующего столбца);

(ii) ***default <значение по умолчанию>***;

(iii) ***check <условие проверки>***;

(iv) ***unique*** – требование, что бы все значения в столбце были различны;

(v) ***primary key*** – определение первичного ключа для однозначной идентификации записей в таблице.

Использование данных директив иллюстрирует пример, приведенный ниже (БД не может содержать двух одноименных таблиц, предыдущая версия таблицы ***Студент*** должна быть удалена директивой ***drop table***):

```
create table Студент (
   Ном_Зач int primary key,
   ФИО varchar(50) not null,
   Сер_Ном_Пасп char(12) not null unique,
   Гражданство varchar(50) default 'Российская Федерация',
   Адрес varchar(250) null,
   Факультет char(1) not null check (Факультет in ('1', '2', '3', '4', '5')),
   Группа char(5) not null )
go
```

В примере столбец ***Ном_Зач*** является первичным ключом таблицы, директива ***primary key*** автоматически влечет ограничения ***not null*** (значение первичного ключа не может быть неопределенным) и ***unique*** (все значения в столбцах первичного ключа должны быть различны). Столбцы ***ФИО*** и ***Группа*** являются обязательными (не содержат NULL-значений), столбец ***Адрес*** может не заполняться (допускает NULL-значения). Серия и номер паспорта студента (***Сер_Ном_Пасп***) обязательный столбец, кроме того, все значения в нем должны быть различны. Столбец ***Гражданство*** является необязательным, но для него

определено значение по умолчанию, которое будет подставляться, если соответствующее значение не указано при вставке данных в таблицу. Для столбца ***Факультет*** задано условие проверки – значение должно соответствовать одному из пяти элементов множества (оператор ***in*** возвращает логическое значение «истина» или «ложь», в зависимости от того принадлежит или не принадлежит элемент множеству). В общем случае в директиве ***check*** может задаваться логическое условие, в котором используются имя столбца, константы, операции сравнения (>, >=, < …), логические операции (***and***, ***or***, ***not***), рассмотренный выше оператор ***in***, скобки.

СУБД будет отклонять попытки добавления записей в таблицы, если добавляемые значения нарушают ограничения для столбцов, определенные в ***create table***. Директивы ***primary key***, ***unique***, ***check*** могут задаваться как на уровне столбца (в определении столбца), так и на уровне таблицы (в конце ***create table***), в последнем случае они обеспечивают возможности создания составных ключей (включающих более одного столбца), проверки уникальности сочетания значений различных столбцов, совместную проверку значений различных столбцов таблицы.

В приведенном ниже примере:

```
create table Студент (
   Ном_Зач int primary key,
   ФИО varchar(50) not null,
   Сер_Пасп char(5) not null,
   Ном_Пасп char(6) not null,
   Факультет char(1) not null check (Факультет in ('1', '2', '3', '4', '5')),
   Группа char(5) not null,
   unique (Сер_Пасп, Ном_Пасп),
   check (Факультет = left(Группа,1)) )
go
```

директива ***unique*** использована для обеспечения уникальности значений столбцов ***Сер_Пасп*** и ***Ном_Пасп***, таким образом, серии и номера паспортов, хранимые в отдельных столбцах таблицы, могут совпадать у различных студентов, но их сочетание должно быть уникальным в каждой из записей. Для полей

Факультет и ***Группа*** проверяется, что бы первый символ в столбце ***Группа*** совпадал с номером факультета.

Составные первичные ключи используются, когда в таблице нет столбца, значения которого обеспечивают уникальную идентификацию строк, примером такой таблицы может служить:

```
create table Оценка (
   Ном_Зач int not null,
   Дисциплина varchar(50) not null,
   Оценка tinyint not null,
   primary key (Ном_Зач, Дисциплина) )
go
```

Студент получает оценки по разным дисциплинам, по одной дисциплине получают оценки много студентов, таким образом, только сочетание значений в столбцах ***Ном_Зач*** и ***Дисциплина*** позволяет произвести однозначную идентификацию записи.

Приведенный пример иллюстрирует возможность использования составного ключа, вместе с тем, при создании таблиц следует стремиться к тому, что бы ключи были как можно короче, поэтому использование столбца с типом данных varchar(50) в качестве ключевого не является удачным. В таких случаях вводят специальный (системный, «суррогатный») ключ. Так как значения данных ключей не имеют смысла с точки зрения предметной области и используются только в целях идентификации записей, организации ссылок, современные СУБД предлагают механизм для их автоматического формирования. В MS SQL для этих целей реализована директива ***identity [(<начальное значение> [, <приращение>])]***, используемая в сочетании с целочисленными типами данных, в том числе с ***decimal*** не содержащим дробной части. Если ***<начальное значение>*** и ***<приращение>*** опущены, в качестве их значений используется 1.

Примером автоматического формирования ключа в таблице ***Оценка*** может служить следующий (предыдущая версия таблицы ***Оценка*** должна быть удалена директивой ***drop table***):

```
create table Оценка (
   Id int identity primary key,
   Ном_Зач int not null,
   Дисциплина varchar(50) not null,
   Оценка tinyint not null )
go
```

1.3 Ссылочная целостность

Большинство современных СУБД поддерживает декларативную ссылочную целостность для обеспечения корректности ссылок между таблицами БД. Ссылочная целостность определяется посредством создания внешних ключей директивой ***foreign key (<столбец 1 ДТ> [, <столбец 2 ДТ> [, …]]) references <родительская таблица> (<столбец 1 РТ> [, <столбец 2 РТ> [, …]])*** в конце оператора ***create table***. Здесь ***<столбец i ДТ>*** – столбец дочерней таблицы (таблицы, из которой осуществляется ссылка), ***<столбец i РТ>*** – столбец родительской таблицы (таблицы, на которую осуществляется ссылка). Кандидатом на внешний ключ в таблице ***Оценка*** является ***Ном_Зач***:

```
create table Оценка (
   Id int identity primary key,
   Ном_Зач int not null,
   Дисциплина varchar(50) not null,
   Оценка tinyint not null,
   foreign key (Ном_Зач) references Студент (Ном_Зач) )
go
```

Если внешний ключ является простым, директива может быть указана в сокращенном виде непосредственно в определении столбца таблицы:

```
create table Оценка (
   Id int identity primary key,
   Ном_Зач int not null references Студент,
   Дисциплина varchar(50) not null,
   Оценка tinyint not null)
go
```

При вставке данных в таблицу, а так же изменении данных, СУБД автоматически контролирует наличие значения внешнего ключа в поле родительской

таблицы, на которое осуществляется ссылка. Если данное значение отсутствует – операция будет отклонена.

Для записей родительской и дочерней таблицы можно задать правила совместной обработки по отношению к операциям удаления и изменения, определив запрет выполнения операции, каскадирование, установку NULL значения или значения по умолчанию – ***on { update | delete } { no action[1] | cascade | set null | set default }[2]***. В таблице:

```
create table Оценка (
   Id int identity primary key,
   Ном_Зач int not null,
   Дисциплина varchar(50) not null,
   Оценка tinyint not null,
   foreign key (Ном_Зач) references Студент (Ном_Зач)
   on delete no action
   on update cascade )
go
```

ограничение ссылочной целостности запрещает удаление записи в таблице ***Студент***, если на соответствующую запись есть ссылки из таблицы **Оценка**, и требует автоматической модификации значений в столбце ***Ном_Зач*** таблицы ***Оценка***, при изменении соответствующих значений в таблице ***Студент***.

1.4 Лабораторная работа 1

По аналогии с примерами, приведенными в п. 1.1 – 1.3, произвести создание БД и таблиц, определив правила проверки значений и задав ограничения ссылочной целостности. Структура БД должна обеспечивать хранение сведений, необходимых для выполнения запросов, указанных в варианте задания.

Содержание отчета:

— схема БД;

— скрипт SQL для создания таблиц;

---

[1] В других диалектах SQL — ***restrict***.

[2] Скобки { | } — указывают на необходимость выбора одной из синтаксических конструкций.

— описание структуры таблиц, ограничений на значения данных, ссылочной целостности, реализованных в БД.

Варианты заданий приведены в ПРИЛОЖЕНИИ.

2. ЗАПОЛНЕНИЕ И МОДИФИКАЦИЯ ТАБЛИЦ БАЗЫ ДАННЫХ

2.1 Вставка данных в таблицы

Вставку данных в таблицы осуществляет команда ***insert into <имя таблицы> [(<столбец 1> [, <столбец 2> [, …]])] values (<значение 1> [, <значение 2> [, …]])***, при этом ***<значение i>*** будет подставлено в ***<столбец i>***. Если значения указываются для всех столбцов таблицы, список столбцов может быть опущен, в данном случае последовательность указания значений должна строго соответствовать последовательности столбцов в операторе ***create table***.

Создадим в БД таблицу:

```
create table Студент (
   Ном_Зач int primary key,
   ФИО varchar(50) not null,
   Факультет tinyint not null,
   Группа char(7) not null )
go
```

Для ее заполнения можно воспользоваться следующими операторами:

```
insert into Студент values (11, 'Лисичкин', 4, '4001')
insert into Студент values (22, 'Сыроежкин', 4, '4001')
insert into Студент values (33, 'Груздев', 4, '4002КФс')
insert into Студент values (44, 'Сморчков', 4, '4002КФс')
insert into Студент values (55, 'Волнушкин', 5, '5001')
insert into Студент values (77, 'Строчков', 5, '5001')
insert into Студент values (88, 'Белов', 5, '5002К')
insert into Студент values (87, 'Краснов', 5, '5002К')
go
```

Создадим в БД таблицу:

```
create table Оценка (
   Id int identity primary key,
   Ном_Зач int not null,
   Дисциплина varchar(50) not null,
   Оценка tinyint not null,
   foreign key (Ном_Зач) references Студент (Ном_Зач)
   on delete cascade
   on update cascade )
go
```

Для ее заполнения можно воспользоваться следующими операторами:

```
insert into Оценка (Ном_Зач, Дисциплина, Оценка) values (11, 'БД', 5)
insert into Оценка (Ном_Зач, Дисциплина, Оценка) values (11, 'ФиЛП', 4)
insert into Оценка (Ном_Зач, Дисциплина, Оценка) values (22, 'БД', 4)
insert into Оценка (Ном_Зач, Дисциплина, Оценка) values (22, 'ФиЛП', 4)
insert into Оценка (Ном_Зач, Дисциплина, Оценка) values (33, 'БД', 3)
insert into Оценка (Ном_Зач, Дисциплина, Оценка) values (33, 'ФиЛП', 4)
insert into Оценка (Ном_Зач, Дисциплина, Оценка) values (44, 'БД', 4)
insert into Оценка (Ном_Зач, Дисциплина, Оценка) values (44, 'ФиЛП', 3)
insert into Оценка (Ном_Зач, Дисциплина, Оценка) values (55, 'БД', 4)
insert into Оценка (Ном_Зач, Дисциплина, Оценка) values (55, 'АСУ', 5)
insert into Оценка (Ном_Зач, Дисциплина, Оценка) values (77, 'БД', 5)
insert into Оценка (Ном_Зач, Дисциплина, Оценка) values (77, 'АСУ', 4)
insert into Оценка (Ном_Зач, Дисциплина, Оценка) values (88, 'БД', 3)
insert into Оценка (Ном_Зач, Дисциплина, Оценка) values (88, 'АСУ', 3)
insert into Оценка (Ном_Зач, Дисциплина, Оценка) values (87, 'БД', 5)
insert into Оценка (Ном_Зач, Дисциплина, Оценка) values (87, 'АСУ', 5)
go
```

При вставке данных значения полей ***identity*** не указываются. Значение, присвоенное данному полю можно получить, опросив глобальную системную переменную ***@@ identity***:

```
select @@identity
go
```

Для просмотра данных в таблице можно воспользоваться простейшим вариантом оператора выборки данных: ***select * from <имя таблицы>***:

```
select * from Студент
select * from Оценка
go
```

Вставка данных в таблицы возможна по результатам выборки из других таблиц оператором ***insert into <имя таблицы> [(<столбец 1> [, <столбец 2> [, …]])] select <выражение 1> [, < выражение 2> [, …]] from <имя таблицы 1> [, <имя таблицы 2> [, …]] [where <условие>]***. В простейшем случае ***< выражение i>*** — это имя столбца.

Создадим в БД таблицу:

```
create table Группа (
   Id smallint identity primary key,
   Номер char(7) not null,
   Факультет tinyint not null )
go
```

для ее заполнения можно воспользоваться следующим оператором:

```
insert into Группа (Номер, Факультет)
   select distinct Группа, Факультет from Студент
go
```

Конструкция ***distinct*** обеспечивает исключение повторяющихся строк при выборке из таблиц.

2.2 Изменение данных

Изменение данных в таблице осуществляет оператор ***update <имя таблицы> set <имя столбца 1> = <выражение 1> [, < имя столбца 2> = < выражение 2> [, …]] [where <условие>]***. Здесь ***<условие>*** — это логическое выражение, построенное из имен столбцов, констант, операций сравнения (>, <, = и др.), логических операции (***and***, ***or***, ***not***), оператора проверки принадлежности множеству (***in***), скобок и некоторых других конструкций. Оператор ***update*** модифицирует все записи таблицы, для которых оказалось истинным ***<условие>***. Если раздел ***where*** опущен – будут изменены все записи таблицы.

Оператор:

```
update Оценка
   set
         Оценка = 4
   where
         Ном_зач = 87 and Дисциплина = 'БД'
go
```

изменит значение оценки, полученной указанным студентом по заданной дисциплине.

Оператор:

```
update Студент
   set
         Факультет = 3,
         Группа = '3002К'
   where
         Группа = '5002К'
go
```

осуществит перевод студентов группы 5002К на 3-й факультет.

2.3 Удаление данных

Для удаления данных в таблице используется директива ***delete [from] <имя таблицы> [where <условие>]***. Здесь ***<условие>*** играет ту же роль, что и в операторе ***update***.

Оператор:

```
delete from Оценка
   where
         Ном_зач = 87 and Дисциплина = 'БД'
go
```

удалит информацию о сдаче экзамена указанным студентом по заданной дисциплине.

Если в таблице, в которую осуществляется вставка из другой таблицы, присутствует столбец со свойством ***identity***, для сохранения существующих ключей можно отключить генерацию значений ***identity*** с помощью директивы ***set identity_insert <имя таблицы> { on | off }***. В MS SQL в один момент времени только

для одной таблицы в сеансе может быть установлено ***on***, при необходимости установить ***on*** для другой таблицы, необходимо предварительно выполнить сброс – ***off*** для текущей.

2.4 Изменение определения таблицы

Таблица, созданная в БД (например, с помощью ***create table***) существует, пока не будет явно удалена (например, с помощью ***drop table***). Так как при удалении таблицы происходит удаление всех содержащихся в ней записей, в языке SQL предусмотрен оператор ***alter table***, позволяющий осуществить модификацию таблицы сохранив, если это возможно, содержащиеся в ней сведения.

Основные варианты синтаксиса и действий, выполняемых с помощью ***alter table***:

(i) ***alter table <имя таблицы> add <определение столбца>*** — добавление столбца к таблице, например:

```
alter table Студент add Ср_балл real
go
```

(ii) ***alter table <имя таблицы> drop column[1] <имя столбца>*** — удаление столбца таблицы, например:

```
alter table Студент drop column Ср_балл
go
```

(iii) ***alter table <имя таблицы> alter column[2] <определение столбца>*** — модификация столбца таблицы[3], например:

---

[1] В других диалектах SQL — ***drop***, ***delete***.
[2] В других диалектах SQL — ***alter***, ***modify***.
[3] Изменение типа данных столбца возможно, если между старым и новым типами существует автоматическое преобразование.

```
alter table Студент alter column ФИО varchar(70) not null
go
```

(iv) ***alter table <имя таблицы> rename <имя столбца> to <новое имя>*** — переименование столбца таблицы, в MS SQL не поддерживается, используется системная хранимая процедура (СХП):

```
sp_rename 'Студент.Группа', 'Номер_группы'
go
```

(v) ***alter table <имя таблицы> rename <новое имя>*** — переименование таблицы, в MS SQL не поддерживается, используется СХП:

```
sp_rename 'Оценка', 'Успеваемость'
go
```

(vi) ***alter table <имя таблицы> add constraint <определение ограничения>***[1] — добавление ограничения к таблице, например:

```
alter table Группа add constraint Уник_номер unique (Номер)
go
```

(vii) ***alter table <имя таблицы> drop constraint <имя ограничения>***[2] — удаление ограничения таблицы, например:

```
alter table Группа drop constraint Уник_номер
go
```

В качестве ограничений могут так же выступать помимо ***unique*** директивы ***primary key***, ***foreign key***, ***check***, ***default***. В приведенных примерах ***Уник_номер*** — имя ограничения. Если имя ограничения не указывается в ***create table*** или ***alter table***, SQL-сервер присваивает ему автоматически сформированное имя. SQL-сервер может отклонить создание ограничения, если ему не соответствуют данные, ранее занесенные в таблицы.

Воспользуемся приведенными конструкциями для изменения текущей БД, зададим уникальность для номера группы:

---

[1] В других диалектах SQL допускается — ***add primary key…*** .
[2] В других диалектах SQL допускается — ***drop primary key***.

```
alter table Группа add constraint Уник_номер unique (Номер)
go
```

Повторно заполним таблицу ***Группа***:

```
delete from Группа
go
insert into Группа (Номер, Факультет)
   select distinct Номер_группы, Факультет from Студент
go
```

Добавим в ***Студент*** столбец ***Группа***:

```
alter table Студент add Группа smallint
go
```

и заполним его:

```
update Студент
   set Группа =
         (select Id from Группа
               where Группа.Номер = Студент.Номер_группы)
go
```

удалим столбец ***Номер_группы***:

```
alter table Студент drop column Номер_группы
go
```

Удалим столбец ***Факультет***:

```
alter table Студент drop column Факультет
go
```

Определим столбец ***Группа*** как внешний ключ:

```
alter table Студент add constraint Вн_кл_группа
   foreign key (Группа) references Группа (Id)
   on delete set null
   on update cascade
go
```

Зададим уникальность для номера зачетной книжки:

```
alter table Студент add constraint Уник_ном_зач unique (Ном_Зач)
go
```

Создадим справочник дисциплин:

```
create table Дисциплина (
   Id smallint identity primary key,
   Наименование varchar(50) not null constraint Уник_наимен
unique )
go
```

Заполним справочник дисциплин:

```
insert into Дисциплина (Наименование)
   select distinct Дисциплина from Успеваемость
go
```

Добавим в ***Успеваемость*** столбец ***Id_Дисциплины***:

```
alter table Успеваемость add Id_Дисциплины smallint
go
```

и заполним его:

```
update Успеваемость
   set Id_Дисциплины =
         (select Id from Дисциплина
               where Дисциплина.Наименование =
                     Успеваемость.Дисциплина)
go
```

Удалим столбец ***Дисциплина***:

```
alter table Успеваемость drop column Дисциплина
go
```

Определим столбец ***Id_Дисциплины*** как внешний ключ:

```
alter table Успеваемость add constraint Вн_кл_дисциплина
   foreign key (Id_Дисциплины) references Дисциплина (Id)
   on delete no action
   on update cascade
go
```

Определим столбец ***Id_Дисциплины*** как обязательный:

```
alter table Успеваемость alter column Id_Дисциплины smallint not
null
go
```

Зададим ограничение уникальности (предполагается, что дисциплины изучаются один семестр и не может быть 2-х оценок по одной дисциплине):

```
alter table Успеваемость add constraint Уник_ном_зач_id_дисц
   unique (Ном_Зач, Id_Дисциплины)
go
```

Зададим ограничение на значение оценки (фиксируются только положительные оценки, истории двоек и не аттестации не ведется):

```
alter table Успеваемость add constraint Полож_оценка
   check (Оценка in (3, 4, 5))
go
```

Для просмотра результирующей схемы БД необходимо выполнить команду ***Создать диаграмму базы данны/New Database Diagram*** из контекстного меню узла ***Диаграммы баз данны/Database Diagrams*** в обозревателе объектов.

Для получения текста SQL для создания таблицы можно выполнить команды ***Создать сценарий для таблицы\Используя CREATE\Новое окно редактора запросов/Script Table as\CREATE to\New Query Editor Window*** из контекстного меню узла, соответствующего таблице в обозревателе объектов.

Для получения полного текста SQL для создания БД необходимо выполнить команды ***Задачи\Сформировать сценарии…/Tasks\Generate Scripts…*** из контекстного меню узла, соответствующего базе данных в обозревателе объектов.

2.5 Лабораторная работа 2

Выполнить вставку тестовых данных в таблицы, созданные в ходе выполнения лабораторной работы 1. В строках, вставляемых в таблицы, должны быть данные как удовлетворяющие, так и не удовлетворяющие условиям запросов, приведенных в варианте задания. В случае внесения в таблицы ошибочных данных произвести их корректировку операторами ***update*** и ***delete***. При

обнаружении недочетов в структуре БД произвести ее корректировку с помощью ***alter table***.

Содержание отчета:

— схема БД (если изменялась);

— наборы данных, содержащихся в таблицах БД;

— примеры использования ***insert***, ***update*** и ***delete*** для корректных и некорректных данных (нарушающих ограничения и ссылочную целостность);

— примеры ***update*** и ***delete***, вызывающих каскадные изменения и удаление данных;

— примеры использования ***alter table*** для корректировки структуры таблиц.

Варианты заданий приведены в ПРИЛОЖЕНИИ.

3. ЗАПРОСЫ НА ЯЗЫКЕ SQL: ВЫБОРКА ДАННЫХ

3.1 Оператор select

В простейшем случае синтаксис оператора для выборки данных из таблиц выглядит следующим образом:

```
select  <список выбора>
   from <список таблиц>
   [ where <условие> ]
go
```

Здесь ***<список выбора>*** — список столбцов таблиц или выражений с их участием, значения которых должен вернуть оператор, если ***<список выбора>*** содержит более одного элемента, они разделяются запятыми, для выборки всех столбцов указывается *; ***<список таблиц>*** содержит перечень таблиц, из которых осуществляется выборка, если в списке более одного элемента, они разделяются запятыми, в разделе ***from*** могут использоваться также представления и производные таблицы; ***<условие>*** аналогично условиям, рассмотренным в операторах ***delete*** и ***update***, в условиях могут также использоваться подзапросы. Если

<условие> опущено, возвращаются все строки, в противном случае только те, для которых оно оказалось истинным.

Оператор ***select*** реализует в языке SQL все операторы, предусмотренные реляционной алгеброй (РА). Пусть в БД существуют таблицы, созданные и модифицированные так, как описано в лабораторной работе 2, а именно таблица ***Группа***:

Id	Номер	Факультет
7	3002К	3
8	4001	4
9	4002КФс	4
10	5001	5

таблица ***Студент***:

Ном_Зач	ФИО	Группа
11	Лисичкин	8
22	Сыроежкин	8
33	Груздев	9
44	Сморчков	9
55	Волнушкин	10
77	Строчков	10
87	Краснов	7
88	Белов	7

таблица ***Дисциплина***:

Id	Наименование
1	АСУ
2	БД
3	ФиЛП

таблица ***Успеваемость***:

Id	Ном_Зач	Оценка	Id_Дисциплины
1	11	5	2
2	11	4	3
3	22	4	2
4	22	4	3
5	33	3	2

6	33	4	3
7	44	4	2
8	44	3	3
9	55	4	2
10	55	5	1
11	77	5	2
12	77	4	1
13	88	3	2
14	88	3	1
16	87	5	1

тогда операцию выборки списка групп 4-го факультета, осуществит оператор:

```
select  * from Группа where Факультет = 4
go
```

проекцию, возвращающую список всех групп, выполнит оператор:

```
select  Номер from Группа
go
```

соединение, формирующее список групп и студентов, реализует оператор:

```
select  * from Группа, Студент
   where Id = Группа
go
```

оператор:

```
select    Номер  'Группа',  ФИО,  Наименование  'Дисциплина',
    Оценка
   from  Группа, Студент, Дисциплина, Успеваемость
   where        Группа.Id = Группа and
            Студент.Ном_Зач = Успеваемость.Ном_Зач and
            Дисциплина.Id = Id_Дисциплины and
            Номер = '4001'
go
```

использует комбинацию соединения выборки и проекции, что бы вернуть список оценок студентов группы 4001. Как следует из примера, в случае, если столбцы различных таблиц имеют одинаковые имена, для уточнения в качестве префикса может использоваться имя таблицы, отделяемое от имени столбца точкой. Для удобства восприятия пользователем набора данных, возвращаемого

запросом, столбцам можно присваивать псевдонимы, используя директиву ***as*** или просто разделяя пробелом имя столбца (выражение) и псевдоним. Если заданы, псевдонимы подставляются в заголовок набора данных, возвращаемого запросом.

По умолчанию для списка выбора действует директива ***all***, поэтому запрос «По каким дисциплинам были выставлены оценки?»:

```
select  Наименование 'Есть оценки'
   from  Дисциплина, Успеваемость
   where        Дисциплина.Id = Id_Дисциплины
go
```

вернет каждую дисциплину столько раз, сколько записей встречается в таблице ***Успеваемость***, а запрос:

```
select  distinct Наименование 'Есть оценки'
   from  Дисциплина, Успеваемость
   where        Дисциплина.Id = Id_Дисциплины
go
```

с помощью директивы ***distinct*** исключит повторяющиеся значения.

Роль, аналогичную операции переименования атрибутов РА, играют псевдонимы таблиц, указываемые в разделе ***from*** и отделяемые от имени таблицы директивой ***as*** или пробелом. Псевдонимы необходимы, например, при выполнении соединения таблицы с собой. Запрос «Кто из студентов получил одинаковые оценки по БД и по ФиЛП?», может быть реализован в SQL с использованием псевдонимов:

```
select  ФИО
   from  Студент, Дисциплина Д1, Дисциплина Д2,
         Успеваемость У1, Успеваемость У2
   where        Д1.Наименование = 'БД' and Д2.Наименование =
'ФиЛП' and
         Д1.Id = У1.Id_Дисциплины and Д2.Id = У2.Id_Дисци-
   плины and
         У1.Ном_Зач = Студент.Ном_Зач and
         У2.Ном_Зач = Студент.Ном_Зач
go
```

В примере две записи из таблицы ***Дисциплина*** рассматриваются под псевдонимами ***Д1*** и ***Д2***, а две связанные с ними записи таблицы ***Успеваемость*** под псевдонимами ***У1*** и ***У2***, после чего выполняется их соединение.

Следует учитывать, что при использовании двух псевдонимов для одной таблицы, каждая запись может быть «выбрана» вначале под первым, а затем под вторым. Данную ситуацию иллюстрирует запрос «Какие студенты получили одинаковые оценки по БД?», если в выражении ***С1.Ном_Зач > С2.Ном_Зач*** заменить **>** на **< >**, каждая пара студентов войдет в результирующий набор данных дважды:

```
select  С1.ФИО, С2.ФИО, У1.Оценка
   from  Студент С1, Студент С2, Дисциплина,
         Успеваемость У1, Успеваемость У2
   where        Наименование = 'БД' and
         Дисциплина.Id = У1.Id_Дисциплины and
         Дисциплина.Id = У2.Id_Дисциплины and
         У1.Оценка = У2.Оценка and
         У1.Ном_Зач = С1.Ном_Зач and У2.Ном_Зач =
    С2.Ном_Зач and
         С1.Ном_Зач > С2.Ном_Зач
go
```

Для удобства восприятия пользователем набора данных, возвращаемого запросом, он может быть отсортирован. Порядок сортировки задается директивой ***order by <столбец 1> [{ asc | desc }] [, <столбец 2> [{ asc | desc }] [, …]]***, здесь ***asc*** задает порядок сортировки по возрастанию (действует по умолчанию), ***desc*** — по убыванию. Пример использования сортировки показывает запрос:

```
select  С1.ФИО, С2.ФИО, У1.Оценка
   from  Студент С1, Студент С2, Дисциплина,
         Успеваемость У1, Успеваемость У2
   where        Наименование = 'БД' and
         Дисциплина.Id = У1.Id_Дисциплины and
         Дисциплина.Id = У2.Id_Дисциплины and
         У1.Оценка = У2.Оценка and
         У1.Ном_Зач = С1.Ном_Зач and У2.Ном_Зач =
    С2.Ном_Зач and
         С1.Ном_Зач > С2.Ном_Зач
```

```
    order by У1.Оценка desc, С1.ФИО, С2.ФИО
go
```

3.2 Директивы, используемые в условиях запросов

В операторе ***select*** можно использовать директиву ***[not] in*** для проверки принадлежности или не принадлежности множеству, следующий оператор вернет всех студентов, за исключением обучающихся в группах 4001, 5001:

```
select  Номер 'Группа', ФИО
    from  Группа, Студент
    where         Группа.Id = Группа and
            Номер not in ( '4001', '5001')
go
```

Для неточного сравнения строк используется директива ***[not] like***, запрос:

```
select  * from Студент
    where ФИО like 'С%'
go
```

вернет информацию о студентах, фамилии которых начинаются с буквы ***С***. В шаблоне, указываемом в операторе ***like*** можно использовать знак ***%*** для обозначения любой, в том числе пустой последовательности символов и знак _ для обозначения произвольного символа (символ должен присутствовать в строке). Если ***%*** или _ содержится в искомой строке, определяется символ ***escape***, например условие ***like '#%%' escape '#'*** определяет символ #, как отменяющий действие символа ***%***, в результате будут найдены строки, начинающиеся с данного символа.

С помощью ***[ ]*** в like можно задавать множество допустимых символов, например выборка студентов с ФИО, начинающейся с ***К*** или ***С***:

```
select  * from Студент
    where ФИО like '[КС]%'
go
```

А с помощью ^ можно определять не вхождение символа в шаблон, например студенты, фамилии которых начинаются с буквы ***С***, где вторая буква не ***ы***:

```
select  * from Студент
   where ФИО like 'С[^ы]%'
go
```

Для удобства составления запросов, производящих отбор диапазонов значений, используется директива ***[not] between <нижняя граница> and <верхняя граница>***. Пример использования директивы иллюстрирует запрос:

```
select  * from Группа
   where Номер between '4000' and '5000'
go
```

В случае если при вставке строки в таблицу не указывалось значение какого-либо столбца, в качестве его значения подставляется специальная константа ***null***. Константа ***null*** не соответствует нулю или пустой строке, она говорит о том, что значение является неопределенным. Как следствие, любая операция сравнения с ***null*** -значением дает неопределенный результат, для поиска в таблице значений отличных или совпадающих с ***null*** используется директива ***is [not] null***.

3.3 Лабораторная работа 3

По аналогии с примерами, приведенными в п. 3.1, 3.2:

— реализовать запросы а) .. в), указанные в варианте задания, без использования подзапросов, агрегатных функций и теоретико-множественных операций;

— самостоятельно предложить и реализовать запросы, демонстрирующие использование директив ***distinct***, ***order by***, ***as***, ***[not] in***, ***[not] between … and …***, ***is [not] null***, ***[not] like***.

Содержание отчета:

— текст запросов на SQL (с пояснениями/комментариями);

— наборы данных, возвращаемые запросами.

Варианты заданий приведены в ПРИЛОЖЕНИИ.

4. ЗАПРОСЫ НА ЯЗЫКЕ SQL: АГРЕГАТНЫЕ ФУНКЦИИ

4.1 Агрегатные функции

Часто результатом выполнения запроса должны быть не сами данные таблиц, а результаты их обработки. Для решения таких задач в языке SQL используются агрегатные функции:

— ***count*** – подсчет количества значений;

— ***sum*** – сумма значений;

— ***avg*** – вычисление среднего;

— ***min*** – поиск минимального значения;

— ***max*** – поиск максимального значения.

MS SQL поддерживает также ряд других функций, полный их перечень приведен в [https://docs.microsoft.com/ru-ru/sql/t-sql/functions/aggregate-functions-transact-sql?view=sql-server-2017].

Определить количество студентов в университете можно, воспользовавшись следующим запросом:

```
select  count(*) from Студент
go
```

количество групп на четвертом факультете – запросом:

```
select  count(Id) from Группа
   where Факультет = 4
go
```

а средний балл по университету – запросом:

```
select avg(Оценка) from Успеваемость
go
```

Часто требуется вычислить значение агрегатной функции, полученное не по всем записям таблицы, а по отдельным группам записей, такую возможность предоставляет директива ***group by***. Вначале осуществляется группировка

записей таблицы по значениям столбцов, указанных в ***group by***, после чего для каждой группы вычисляется агрегатная функция. Запрос:

```
select  Номер, count(Ном_Зач) as 'Количество студентов'
   from Группа, Студент
   where Группа = Id
   group by Номер
go
```

позволит получить количество студентов в каждой из групп, а запрос:

```
select  Номер, ФИО, avg(Оценка) as 'Ср. балл'
   from Группа, Студент, Успеваемость
   where        Группа = Группа.Id and
            Студент.Ном_Зач = Успеваемость.Ном_Зач
   group by Номер, ФИО, Успеваемость.Ном_Зач
   order by Номер, ФИО
go
```

сформирует значение среднего балла для каждого из студентов. Следует заметить, что стандарт языка SQL предполагает, что столбцы, входящие в список выбора, должны входить в список, в соответствии с которым осуществляется группировка, вместе с тем, некоторые СУБД могут отступать от этих требований.

В общем случае использование агрегатной функции предполагает следующий синтаксис ***<имя функции> ([all | distinct] <выражение>)***. Особенности функции ***count***:

— ***count(*)*** подсчитывает все значения, в том числе ***null***;

— ***count(all <выражение>)*** подсчитывает все значения, отличные от ***null***;

— ***count(distinct <выражение>)*** подсчитывает все различные значения, отличные от ***null***.

Использование ***distinct*** с функциями ***min*** и ***max*** допускается, но не имеет смысла. Указание ***distinct*** для функций ***sum*** и ***avg*** приводит к тому, что будут обрабатываться только уникальные значения, отличные от ***null***. Данные функции возвращают ***null***, если в столбце нет других значений.

Аргументом агрегатной функции в общем случае может быть выражение, при этом некоторые SQL сервера допускают, что бы аргументом была другая агрегатная функция.

Конструкция ***having*** играет ту же роль по отношению к группам записей, что и ***where*** для отдельных строк и позволяет наложить условие на значение, сформированное агрегатной функцией. Для получения перечня групп, в которых числится 25 и более студентов можно воспользоваться запросом:

```
select  Номер, count(Ном_Зач) as ' >= 25 студентов'
   from  Группа, Студент
   where         Группа = Id
   group by Номер
   having count(Ном_Зач) >= 25
go
```

а для нахождения групп, в которых есть студенты со средним баллом 5.0 – запросом:

```
select  Номер, ФИО
   from  Группа, Студент, Успеваемость
   where         Группа = Группа.Id and
            Студент.Ном_Зач = Успеваемость.Ном_Зач
   group by Номер, ФИО, Успеваемость.Ном_Зач
   having avg(Оценка) = 5
   order by Номер, ФИО
go
```

В общем случае, запрос, содержащий ***where***, ***group by***, ***having*** и агрегатные функции выполняется в следующей последовательности:

(i) отбираются записи таблицы в соответствии с условием, заданным во WHERE;

(ii) записи группируются в соответствии с порядком, определенным в GROUP BY;

(iii) для каждой группы записей вычисляется значение агрегатной функции;

(iv) на полученные значения накладывается условие отбора, определенное в HAVING.

4.2 Объединение, пересечение, разность запросов

С помощью директив ***union***, ***intersect***, ***except*** можно осуществлять операции объединения, пересечения и разности запросов, аналогичные соответствующим операциям РА над отношениями. Следующий запрос вернет общие дисциплины, изучаемые как студентами 4-го, так и 5-го факультетов (пересечение):

```
select  distinct Наименование
   from  Группа, Студент, Успеваемость, Дисциплина
   where        Группа = Группа.Id and
            Студент.Ном_Зач = Успеваемость.Ном_Зач and
            Id_Дисциплины = Дисциплина.Id and
            Факультет = '4'
intersect
select  distinct Наименование
   from  Группа, Студент, Успеваемость, Дисциплина
   where        Группа = Группа.Id and
            Студент.Ном_Зач = Успеваемость.Ном_Зач and
            Id_Дисциплины = Дисциплина.Id and
            Факультет = '5'
go
```

Для получения множества дисциплин, изучаемых на 4-м и не изучаемых на 5-м факультете, в приведенном запросе директиву ***intersect*** следует заменить на ***except***, а для получения общего множества дисциплин — на директиву ***union***. Часто SQL-сервера не реализуют ***intersect*** и ***except***, так как соответствующая обработка может быть выполнена с помощью экзистенциальных запросов (см. лабораторную работу 6).

В общем случае директивы ***union***, ***intersect***, ***except*** могут быть использованы в последовательности из нескольких запросов, при этом следует учитывать следующее:

(i) во всех запросах должно совпадать количество элементов в списках выбора;

(ii) запросы должны возвращать элементы, имеющие совместимые типы данных;

(iii) в заголовке результирующего набора данных будут использованы имена элементов или псевдонимы (если заданы), указанные в списке выбора первого запроса.

Следует отметить, что SQL-сервера, как правило, реализуют как теоретико-множественные операции объединения, пересечения и разности, так и мульти-множественные. Пусть в отношении ***r*** кортеж ***t*** встречается ***n*** раз, а в отношении ***s*** — ***m*** раз, тогда:

(i) объединение ***r*** и ***s*** будет содержать ***n*** + ***m*** вхождений кортежа ***t***;

(ii) пересечение ***r*** и ***s*** будет содержать ***min(n, m)*** вхождений кортежа ***t***;

(iii) разность ***r*** и ***s*** будет содержать ***max(0, n – m)*** вхождений кортежа ***t***.

Способ выполнения операции теоретико-множественная или мульти-множественная задается директивами ***distinct*** (действует по умолчанию) и ***all*** [1] соответственно.

4.3 Лабораторная работа 4

По аналогии с примерами, приведенными выше:

— реализовать запросы г) .. е), указанные в варианте задания;

— самостоятельно предложить и реализовать запросы с агрегатными функциями, не использованными при выполнении варианта задания;

— самостоятельно предложить и реализовать запросы, демонстрирующие применение каждой из директив, приведенных в п. 4.2., показать различия в получаемых результатах при выполнении теоретико-множественных операций и операций над мультимножествами.

Содержание отчета:

— текст запросов на SQL (с пояснениями/комментариями);

— наборы данных, возвращаемые запросами.

Варианты заданий приведены в ПРИЛОЖЕНИИ.

---

[1] В MS SQL ***all*** не поддерживается в сочетании с ***intersect*** и ***except*** , поддерживается, например, в SQLA [EXCEPT Statement - SAP Help Portal]

5. ЗАПРОСЫ НА ЯЗЫКЕ SQL: ПОДЗАПРОСЫ

5.1 Запросы с подзапросам

Сложные операции выборки иногда невозможно выполнить с помощью одного запроса, для решения таких задач в разделах ***where*** и ***having*** могут использоваться подзапросы. При этом подзапрос, в свою очередь, может содержать подзапросы.

Подзапросы часто используются совместно с оператором ***in*** для формирования набора данных, на основе которого внешний запрос проверяет некоторое условие. Предположим, что требуется найти студентов, у которых нет оценок 3, запрос:

```
select  Номер, ФИО
   from  Группа, Студент, Успеваемость
   where        Группа = Группа.Id and
            Студент.Ном_Зач = Успеваемость.Ном_Зач and
            Оценка <> 3
go
```

не даст требуемого результата, так как наличие хороших оценок по какому-либо предмету не исключает наличия троек по другим, решить задачу позволит запрос с подзапросом:

```
select  Номер, ФИО
   from  Группа, Студент
   where        Группа = Группа.Id and
            Студент.Ном_Зач not in
                    (select  Студент.Ном_Зач
                            from  Студент, Успеваемость
                            where        Студент.Ном_Зач    =    Успеваемость.Ном_Зач
                                    and Оценка = 3)
   go
```

В приведенном примере подзапрос формирует множество номеров зачеток студентов, у которых есть тройки по каким-либо дисциплинам, после чего внешний запрос отбирает данные, не принадлежащие этому множеству.

Теоретически уровень вложенности подзапросов ничем не ограничен, но в конкретных реализациях SQL-серверов ограничения могут накладываться. При выполнении запроса с подзапросами вначале выполняется самый вложенный запрос, затем запрос предыдущего уровня и т.д. вплоть до основного запроса.

Подзапросы могут возвращать значения агрегатных функций, при этом, если подзапрос используется с операцией сравнения, он должен возвращать единственное значение. Следующий запрос вернет студентов, имеющих средний бал выше, чем средний балл по университету:

```
select  Номер, ФИО
   from  Группа, Студент, Успеваемость
   where         Группа = Группа.Id and
            Студент.Ном_Зач = Успеваемость.Ном_Зач
   group by Номер, ФИО, Успеваемость.Ном_Зач
   having avg(Оценка) > (select avg(Оценка) from Успеваемость)
   order by Номер, ФИО
go
```

Подзапросы могут использоваться в операторах манипулирования данными: в ***insert*** для вставки данных в таблицы из других таблиц; в ***update*** и ***delete*** для формирования критериев выполнения операций и вычисления новых значений.

Создадим и заполним таблицу ***Дипл_с_отл*** данными о студентах, имеющих средний балл выше 4.75:

```
create table Дипл_с_отл (Группа char(7), ФИО varchar(70),
Ном_зач int)
go

insert into Дипл_с_отл (Группа, ФИО, Ном_Зач)
   select  Номер, ФИО, Студент.Ном_Зач
            from  Группа, Студент, Успеваемость
            where         Группа = Группа.Id and
                     Студент.Ном_Зач = Успеваемость.Ном_Зач
            group by Номер, ФИО, Студент.Ном_Зач
            having avg(Оценка) >= 4.75
            order by Номер, ФИО

go
```

Удалим из нее записи о студентах, имеющих тройки:

```
delete from Дипл_с_отл
   where        Ном_Зач in
          (select  Студент.Ном_Зач
                   from  Студент, Успеваемость
                   where        Студент.Ном_Зач        =        Успевае-
                          мость.Ном_Зач and Оценка = 3)

go
```

Добавим столбец ***Ср_Балл*** и заполним его:

```
alter table Дипл_с_отл add Ср_Балл real
go

update Дипл_с_отл
   set Ср_Балл =
          (select avg(Оценка) from Успеваемость
           where Дипл_с_отл.Ном_Зач = Успеваемость.Ном_Зач)
go
```

Особенности использования подзапросов:

(i) список выбора в подзапросе, используемом в разделах ***where*** и ***having*** должен содержать один элемент;

(ii) подзапросы могут использоваться в разделах ***select***, ***from***, ***where***, ***having***;

(iii) подзапросы не могут использоваться в разделах ***order by***, ***group by***;

(iv) если подзапрос используется с операцией сравнения, он должен возвращать единственное значение.

5.2 Производные таблицы и представления

Подзапросы в разделе ***from*** используются для формирования производных таблиц, содержащих промежуточные результаты, используемые основным запросом. В предыдущем разделе был рассмотрен запрос, возвращающий группы, число студентов в которых превышает заданный порог:

```
select  Номер, count(Ном_Зач) as ' >= 25 студентов'
   from  Группа, Студент
   where        Группа = Id
```

```
   group by Номер
   having count(Ном_Зач) >= 25
go
```

альтернативной реализацией для него могла бы быть следующая:

```
select Номер, Кол_во as ' >= 25 студентов'
   from
         (select  Номер, count(Ном_Зач) as 'Кол_во'
               from  Группа, Студент
               where        Группа = Id
               group by Номер) as Кол_во_студ_в_гр
   where  Кол_во >= 25
go
```

Здесь подзапрос в разделе ***from*** формирует производную таблицу, которой присваивается псевдоним ***Кол_во_студ_в_гр***, после чего из нее осуществляется выборка. Определение производной таблицы и ее данные существуют только в течение времени выполнения внешнего запроса. Альтернативный механизм предоставляют представления:

```
create view Кол_во_студ_в_гр (Номер, Кол_во)
as
   select  Номер, count(Ном_Зач)
         from  Группа, Студент
         where        Группа = Id
         group by Номер
go
```

После создания представления директивой ***create view*** его определение, как и определения других объектов БД, существует, пока не будет явно удалено директивой ***drop view***. Данные, возвращаемые представлением, не хранятся в БД, они формируются динамически запросом, на котором основано представление в момент обращения к нему:

```
select Номер, Кол_во as ' >= 25 студентов'
   from Кол_во_студ_в_гр
   where  Кол_во >= 25
go
```

5.3 Экзистенциальные запросы

К особому классу относятся запросы с квантором ***exists***, получившие название экзистенциальных, они имеют следующий синтаксис: ***{ where | having } [not] exists (<подзапрос>)***. Квантор ***exists*** возвращает логическое значение «истина», если сформированный подзапросом набор данных не пуст и «ложь» в противном случае, ***not*** инвертирует значение, возвращаемое квантором.

Используя кванторы можно предложить другой вариант запроса, находящего студентов, у которых нет оценок 3:

```
select  Номер, ФИО
   from  Группа, Студент
   where        Группа = Группа.Id and
          not exists
                (select  *
                      from  Успеваемость
                      where        Студент.Ном_Зач    =    Успеваемость.Ном_Зач
                                  and Оценка = 3)
   go
```

Так как квантор возвращает логическое условие в зависимости от наличия или отсутствия данных, удовлетворяющих условию подзапроса, а сами данные не имеют никакого значения, в списке выбора подзапроса часто используется *.

С помощью экзистенциальных запросов можно реализовать пересечение и разность запросов, так запрос, возвращающий общие дисциплины, изучаемые как студентами 4-го, так и 5-го факультетов (пересечение):

```
select  distinct Наименование
   from  Группа, Студент, Успеваемость, Дисциплина
   where        Группа = Группа.Id and
          Студент.Ном_Зач = Успеваемость.Ном_Зач and
          Id_Дисциплины = Дисциплина.Id and
          Факультет = '4'
intersect
select  distinct Наименование
   from  Группа, Студент, Успеваемость, Дисциплина
   where        Группа = Группа.Id and
          Студент.Ном_Зач = Успеваемость.Ном_Зач and
```

```
                Id_Дисциплины = Дисциплина.Id and
                Факультет = '5'
        go
```

может быть переформулирован следующим образом:

```
        select  distinct Наименование
           from  Группа, Студент, Успеваемость, Дисциплина
           where        Группа = Группа.Id and
                Студент.Ном_Зач = Успеваемость.Ном_Зач and
                Id_Дисциплины = Дисциплина.Id and
                Факультет = '4' and exists
                        (select  *
                                from  Группа Г, Студент С, Успеваемость У,
                                      Дисциплина Д
                                where        С.Группа = Г.Id and
                                      С.Ном_Зач = У.Ном_Зач and
                                      У.Id_Дисциплины = Д.Id and
                                      Факультет = '5' and
                                      Дисциплина.Id = Д.Id)
        go
```

Для реализации разности перед квантором необходимо добавить ***not***.[1] Запросы, реализующие с помощью кванторов теоретико-множественные операции, могут возвращать наборы данных не тождественные наборам данных, возвращаемым запросами с ***intersect*** и ***except*** при наличии в таблицах ***null***-значений. Теоретико-множественные операции обрабатывают ***null***-значения так же, как и любые другие константы. В запросах с ***exists*** используются операции сравнения, которые могут возвращать значение ***unknow***, если одним из операндов является ***null***-значение.

С помощью экзистенциальных запросов можно реализовать оператор деления РА.

Создадим и заполним таблицу ***Д4Ф*** (все дисциплины, изучаемые на 4-м факультете):

---

[1] Запросы могут вернуть различные наборы данных, если в таблице есть ***null*** значения (в отличие от операций сравнения ***union***, ***intersect*** и ***except*** рассматривают ***null*** значения как обычные константы).

```
create table Д4Ф (Д varchar(50))
go

insert into Д4Ф (Д)
   select  distinct Наименование
      from  Группа, Студент, Успеваемость, Дисциплина
      where        Группа = Группа.Id and
               Студент.Ном_Зач = Успеваемость.Ном_Зач and
               Id_Дисциплины = Дисциплина.Id and
               Факультет = '4'
go
```

Создадим и заполним таблицу ***ДФ*** (все факультеты и дисциплины, которые на них изучаются):

```
create table ДФ (Ф tinyint, Д varchar(50))
go

insert into ДФ (Ф, Д)
   select  distinct Факультет, Наименование
      from  Группа, Студент, Успеваемость, Дисциплина
      where        Группа = Группа.Id and
               Студент.Ном_Зач = Успеваемость.Ном_Зач and
               Id_Дисциплины = Дисциплина.Id
go
```

Тогда следующий запрос даст ответ на вопрос, на каких факультетах изучаются все дисциплины из числа изучаемых на факультете 4:

```
select distinct ДФ.Ф from ДФ where not exists
      (select * from Д4Ф where not exists
               (select * from ДФ ДФ1
                        where        ДФ1.Ф = ДФ.Ф and
                                 ДФ1.Д = Д4Ф.Д))
go
```

Для удобства записи запросов в SQL предусмотрены кванторы ***all*** [ALL (Transact-SQL) - SQL Server | Microsoft Docs], ***any*** и ***some*** [SOME | ANY (Transact-SQL) | Документы Майкрософт - SQL Server | Microsoft Docs].

Альтернативой создания таблиц ***Д4Ф*** и ***ДФ*** в предыдущем примере является использование представлений:

```
create view Д4Ф (Д)
as
   select  distinct Наименование
      from  Группа, Студент, Успеваемость, Дисциплина
      where         Группа = Группа.Id and
                Студент.Ном_Зач = Успеваемость.Ном_Зач and
                Id_Дисциплины = Дисциплина.Id and
                Факультет = '4'
go

create view ДФ (Ф, Д)
as
   select  distinct Факультет, Наименование
      from  Группа, Студент, Успеваемость, Дисциплина
      where         Группа = Группа.Id and
                Студент.Ном_Зач = Успеваемость.Ном_Зач and
                Id_Дисциплины = Дисциплина.Id
go
```

5.4 Лабораторная работа 5

По аналогии с примерами, приведенными выше:

— реализовать запросы ж) .. и), указанные в варианте задания, с использованием ***[not] exists*** и подзапросов;

— самостоятельно предложить и реализовать запросы, демонстрирующие использование подзапросов в операторах манипулирования данными (п. 5.1);

— с помощью ***[not] exists*** реализовать запросы, разработанные в п. 4 для иллюстрации использования теоретико-множественных операций, показать различие в выполнении запросов с ***[not] exists*** и теоретико-множественными операциями при наличии в таблицах ***null***-значений (п. 5.3).

Содержание отчета:

— текст запросов на SQL (с пояснениями/комментариями);

— наборы данных, возвращаемые запросами.

Варианты заданий приведены в ПРИЛОЖЕНИИ.

6. ХРАНИМЫЕ ПРОЦЕДУРЫ И ФУНКЦИИ

6.1 Управляющие конструкции в языке SQL

Конструкции, обсуждавшиеся ранее, носили декларативный характер, они задавали критерии выполнения операций, а не алгоритм их выполнения. Вместе с тем в языке SQL присутствуют операторы, соответствующие процедурному подходу, аналогичному принятому в языках программирования высокого уровня (последовательность, ветвление, цикл).

Управляющие конструкции предназначены для использования в:

— пакетных заданиях (ПЗ);

— хранимых процедурах (ХП);

— триггерах (Т).

ХП и Т будут обсуждаться ниже, ПЗ — это последовательность операторов, за которой следует директива ***go***, передающаяся на SQL-сервер и выполняющаяся как один пакет. В качестве примера можно оформить как один пакет создание, заполнение и выборку из таблицы ***Дипл_с_отл***:

```
create table Дипл_с_отл (Группа char(7), ФИО varchar(70),
Ном_зач int)
insert into Дипл_с_отл (Группа, ФИО, Ном_Зач)
   select  Номер, ФИО, Студент.Ном_Зач
           from  Группа, Студент, Успеваемость
           where        Группа = Группа.Id and
                   Студент.Ном_Зач = Успеваемость.Ном_Зач
           group by Номер, ФИО, Студент.Ном_Зач
           having avg(Оценка) >= 4.75
           order by Номер, ФИО
delete from Дипл_с_отл
   where        Ном_Зач in
           (select  Студент.Ном_Зач
                   from  Студент, Успеваемость
                   where        Студент.Ном_Зач        =        Успевае-
                           мость.Ном_Зач and Оценка = 3)
select * from Дипл_с_отл
go
```

В диалекте Transact SQL в качестве разделителя операторов используется перевод строки, MS SQL допускает так же использование в качестве разделителя точки с запятой. SQL-сервера могут накладывать ограничения на использование операторов в ПЗ, типовыми ограничениями являются:

(i) операторы ***create { default | function | procedure | rule | shema | trigger | view }*** нельзя совмещать с другими в одном ПЗ;

(ii) нельзя изменить объект и воспользоваться изменением в одном ПЗ, например следующее ПЗ приведет к ошибке:

```
alter table Дипл_с_отл add Ср_Балл real
update Дипл_с_отл
   set Ср_Балл =
          (select avg(Оценка) from Успеваемость
           where Дипл_с_отл.Ном_Зач = Успеваемость.Ном_Зач)
go
```

К числу управляющих операторов языка SQL относятся:

(i) составной оператор:

```
begin
        <список операторов>
end
```

(ii) условный оператор:

```
if <условие>
        < оператор>
[ else
        < оператор> ]
```

(iii) оператор выбора:

```
case <исходное выражение>
        when <выражение> then <результирующее выражение>
        [ … ]
        [ else <результирующее выражение> ]
end
```

или:

```
case
        when <условное выражение> then <результирующее выражение>
```

```
        [ … ]
        [ else <результирующее выражение> ]
    end
```

первый вариант синтаксиса предполагает, что значение для ***<исходное выражение>*** последовательно сравнивается с ***<выражение>***, при совпадении значений возвращается значение для ***<результирующее выражение>***, расположенное в соответствующей ветви ***when***, если совпадений не найдено, выполняется ветвь ***else***; во втором варианте синтаксиса последовательно вычисляются значения для ***<условное выражение>***, если получена истина, возвращается значение для ***<результирующее выражение>***, расположенное в соответствующей ветви ***when***, если для всех ***<условное выражение>*** получено ложное значение, выполняется ветвь ***else***;

(iv) оператор цикла:

```
    while <условие>
        < оператор>
```

в теле цикла могут использоваться операторы ***break*** для завершения цикла и передачи управления первому оператору за циклом, и ***continue*** для завершения текущей итерации цикла и передачи управления первому оператору в теле цикла;

(v) безусловный переход:

```
    <метка>:
    [ … ]
    go to <метка>
```

(vi) возврат:

```
    return [ <целочисленное выражение> ]
```

(vii) приостановка выполнения на заданный интервал или до наступления момента времени:

```
    waitfor { delay <интервал> | time <время> ]
```

<интервал> и ***<время>*** должны относиться к типу данных ***datetime*** и могут указываться посредством локальных переменных.

В ПЗ, ХП и Т можно объявлять локальные переменные с помощью оператора:

```
declare <имя> <тип данных> [ , … ]
```

имя локальной переменной в качестве префикса должно содержать символ ***@***. Присвоение значений локальным переменным осуществляют операторы:

```
set <имя> = <выражение>
select <имя> = <выражение>
```

второй способ белее универсален, так как некоторые SQL-сервера используют директиву ***set*** для задания свойств сервера и баз данных.

Проиллюстрируем сказанное примерами:

(i) объявление и использование переменных:

```
declare @x int, @y int
set @x = 1
select @y = 2
select @x + @y
go
```

при этом, если далее выполнить ПЗ:

```
select @y
go
```

будет получено сообщение об ошибке, так как после завершения предыдущего ПЗ переменная @y не определена;

(ii) использование ***if*** — поиск претендентов на диплом с отличием:

```
if exists
        (select *
                from  Группа, Студент, Успеваемость
                where         Группа = Группа.Id and
                        Студент.Ном_Зач = Успеваемость.Ном_Зач
                group by Номер, ФИО, Студент.Ном_Зач
                having avg(Оценка) >= 4.75)
begin
        print 'Претенденты на диплом с отличием'
        (select  Номер, ФИО, Студент.Ном_Зач
                from  Группа, Студент, Успеваемость
                where         Группа = Группа.Id and
```

```
                    Студент.Ном_Зач = Успеваемость.Ном_Зач
                group by Номер, ФИО, Студент.Ном_Зач
                having avg(Оценка) >= 4.75)
end
else
        print 'Претендентов на диплом с отличием нет'
go
```

(iii) использование ***case*** — оформление результатов вывода:

```
alter table Успеваемость alter column Оценка tinyint null
go

insert into Успеваемость (Ном_Зач, Оценка, Id_Дисциплины)
values (11, null, 1)
go

select  Номер 'Группа', ФИО, Наименование 'Дисциплина', 'Оцен-
ка' =
        case Оценка
                when 5 then 'Отлично'
                when 4 then 'Хорошо'
                when 3 then 'Удовлетворительно'
                else  'Неизвестно'
        end
        from  Группа, Студент, Дисциплина, Успеваемость
        where         Группа.Id = Группа and
                Студент.Ном_Зач = Успеваемость.Ном_Зач and
                Дисциплина.Id = Id_Дисциплины
go
```

(iv) использование ***while*** — увеличение стипендий студентам на 10%, пока среднее значение стипендии не достигнет заданной величины:

```
alter table Студент add Стипендия smallmoney
go

update Студент
   set Стипендия = Группа * 1000
go
```

```
while (select avg(Стипендия) from Студент) < 10000
   update Студент
         set Стипендия = Стипендия  + Стипендия * 0.1
go
```

(iv) тот же результат с помощью ***goto***:

```
AddMoney:
      update Студент
            set Стипендия = Стипендия  + Стипендия * 0.1
      if (select avg(Стипендия) from Студент) > 10000
            return
      goto AddMoney
go
```

6.2 Хранимые процедуры

ХП — это модуль, состоящий из SQL операторов и находящийся в БД на SQL-сервере, вызываемый клиентскими приложениями. ХП может:

(i) получать и возвращать параметры;

(ii) вызывать другие процедуры;

(iii) возвращать код ошибки (с помощью ***return***);

(iv) возвращать наборы данных, сформированные запросами в теле ХП.

ХП создается оператором, имеющим следующий синтаксис:

```
create { proc | procedure } <имя процедуры> [ ; <номер> ]
      [ @<имя параметра 1> <тип данных 1>
            [ = <значение по умолчанию 1> ] [ { out | output } ] [
      readonly ]
      [ , … ]
      [ with recompile ]
as
      <оператор>
```

<имя процедуры> — идентификатор уникальный в рамках БД;

<номер> — позволяет объединять процедуры в группы, например для возможности удаления всей группы оператором ***drop procedure***, в будущих версиях MS SQL планируется отказаться от этой возможности;

@<имя параметра 1> — идентификатор локального параметра, областью действия которого является тело ХП;

<тип данных 1> — в ХП используется тот же набор типов данных, что и для столбцов таблиц;

<значение по умолчанию 1> — используется, если значение параметра не было указано при вызове ХП;

out или ***output*** говорит о том, что значение параметра является возвращаемым;

readonly — значение параметра не может быть изменено в теле ХП;

recompile — требует перекомпиляции процедуры при каждом ее вызове (например, в целях оптимизации планов выполнения запросов в теле ХП).

ХП вызывается оператором, имеющим следующий синтаксис:

```
[ { exec | execute } [ @<имя переменной> = ] <имя процедуры> [ ; <номер> ]
        [ @<имя параметра 1> ]  = { <значение параметра 1> |
                                    @<имя переменной 1>] [ output ] }
        [ , … ]
        [ with recompile ]
```

@<имя переменной> — имя переменной, которой будут присвоен код возврата, указываемый в операторе ***return*** (признак нормального завершения или код ошибки, возвращаемый процедурой);

<значение параметра 1> — выражение, задающее значение для параметра процедуры;

@<имя переменной 1> — значение параметра задается переменной.

Примеры ХП:

(i) выборка данных:

```
create procedure Гр_4001_5001
as
select * from Группа, Студент where Группа = Id and Номер = '4001'
select * from Группа, Студент where Группа = Id and Номер = '5001'
go
```

```
Гр_4001_5001
go
```

(ii) процедура с параметром:

```
create procedure Гр_N @n char(7) = '4001'
as
select * from Группа, Студент where Группа = Id and Номер = @n
go

Гр_N '5001'
go

Гр_N
go
```

во втором вызове будет использовано значение по умолчанию;

(iii) процедура с возвращаемым параметром:

```
create procedure Кол_во_Студ_в_Гр_N @n char(7) = '4001', @Кол_во
int = 0 out
as
        select @Кол_во = count(*)
                from Группа, Студент
                where Группа = Id and Номер = @n
go

declare @x int
exec Кол_во_Студ_в_Гр_N '5001', @x out
select @x
go
```

Системная ХП (СХП) ***sp_helptext*** выводит текст ХП:

```
sp_helptext 'Кол_во_Студ_в_Гр_N'
go
```

В ХП можно создавать временные таблицы, имя временной таблицы начинается с символа #, временная таблица существует пока не завершено выполнение процедуры, в которой она была создана. Временная таблица доступна в ХП, вызываемых из ХП, в которой она была создана.

Создадим и заполним таблицу ***Подразделение***:

```
create table Подразделение (Id int, Наименование char(20), Подчинено int);
go
insert into Подразделение values (1, 'Гуап', null);
insert into Подразделение values (2, 'Факультет 4', 1);
insert into Подразделение values (3, 'Кафедра 43', 2);
insert into Подразделение values (4, 'Лаборатория 23-10', 3);
insert into Подразделение values (5, 'Факультет 5', 1);
insert into Подразделение values (6, 'Кафедра 53', 5);
insert into Подразделение values (7, 'Лаборатория 23-17', 6);
go
```

Тогда следующая ХП позволит вывести перечень подразделений, входящих в состав заданного:

```
create procedure Орг_стр_ра @Подразд char(20)
as begin
        declare @n int
        select @n = 0
        select @n = Id from Подразделение where Наименование = @Подразд
        create table #Иерарх (Наимен char(20))
        insert into #Иерарх
        select Наименование from Подразделение where Подчинено = @n
        select @n = 0
        while @n <> (select count(*) from #Иерарх)
        begin
                select @n = count(*) from #Иерарх
                insert into #Иерарх
                select П1.Наименование
                        from Подразделение П1, Подразделение П2
                        where        П1.Подчинено = П2.Id and
                                     П2.Наименование in
                                          (select Наимен from #Иерарх) and
                                               П1.Наименование not in
                                               (select Наимен from #Иерарх)
        end
        select * from #Иерарх
end
go

exec Орг_стр_ра 'Факультет 4'
go
```

В процедуре заполняется временная таблица ***#Иерарх***, в которую заносятся подразделения, подчиненные заданному параметром процедуры. Далее организуется цикл, в котором производится добавление в ***#Иерарх*** подразделений, расположенных в иерархии ниже, чем уже присутствующие в ***#Иерарх***. Работа цикла завершается, когда число записей в ***#Иерарх*** перестает расти.

6.3 Хранимые функции

Помимо ХП SQL поддерживает создание функций пользователя (***create function*** [https://docs.microsoft.com/ru-ru/sql/t-sql/statements/create-function-transact-sql?view=sql-server-2017]), которые наряду со стандартными могут использоваться в выражениях и при формировании условий выполнения запросов. Так запрос, выбирающий студентов со средним баллом больше чем средний балл по университету:

```
select  Номер, ФИО
   from  Группа, Студент, Успеваемость
   where        Группа = Группа.Id and
            Студент.Ном_Зач = Успеваемость.Ном_Зач
   group by Номер, ФИО, Успеваемость.Ном_Зач
   having avg(Оценка) > (select avg(Оценка) from Успеваемость)
   order by Номер, ФИО
go
```

может быть реализован с использованием скалярной функции:

```
create function Ср_Балл_Унив () returns real
begin
        declare @Ср_Балл real
        select @Ср_Балл = avg(convert(real, Оценка)) from Успеваемость
        return(@Ср_Балл)
end
go

select  Номер, ФИО
from    Группа, Студент, Успеваемость
where  Группа = Группа.Id and Студент.Ном_Зач = Успеваемость.Ном_Зач
group by Номер, ФИО, Успеваемость.Ном_Зач
having avg(Оценка) > dbo.Ср_Балл_Унив ()
```

```
order by Номер, ФИО
go
```

Хранимые функции могут возвращать табличные значения, приведенная ниже табличная функция возвращает подразделения, непосредственно подчиненные заданному:

```
create function Непоср_Подчин (@Подразд char(20))
   returns table
as
   return (select П2.Id, П2.Наименование
           from Подразделение П1, Подразделение П2
           where
                 П1.Наименование = @Подразд and
                 П2.Подчинено = П1.Id)
go

select * from Непоср_Подчин ('Гуап')
go

select *
   from Подразделение join Непоср_Подчин ('Гуап') as НП
           on Подчинено = НП.Id
go
```

Табличные функции рекомендуется использовать в случаях, когда возвращаемые наборы данных имеют небольшую размерность.

6.4 Лабораторная работа 6

По аналогии с примерами, приведенными в п. 6.1, 6.2, создать в БД ХП, реализующие:

— вставку с пополнением справочников (например, вставляется информация о студенте, если указанный номер группы отсутствует в БД, запись добавляется в таблицу с перечнем групп);

— удаление с очисткой справочников (удаляется информация о студенте, если в его группе нет больше студентов, запись удаляется из таблицы с перечнем групп);

— каскадное удаление (при наличии условия ссылочной целостности ***no action*** перед удалением записи о группе удаляются записи обо всех студентах этой группы);

— вычисление и возврат значения агрегатной функции (на примере одного из запросов из задания);

— формирование статистики во временной таблице (например, для рассматриваемой БД — для каждого факультета: количество групп, количество обучающихся студентов, количество изучаемых дисциплин, средний балл по факультету).

Самостоятельно предложить и реализовать ПЗ или ХП, демонстрирующие использование конструкций, описанных в п. 6.1.

Самостоятельно предложить и реализовать запросы, использующие скалярную и табличную функцию по аналогии с примерами в п. 6.3.

Содержание отчета:

— тексты ХП и их вызовов (с пояснениями/комментариями);

— наборы данных, возвращаемые ХП, ПЗ, функциями, запросами.

Варианты заданий приведены в ПРИЛОЖЕНИИ.

7. ТРИГГЕРЫ

7.1 Виды триггеров

Триггер — это процедура, запускаемая автоматически при модификации данных в таблице (***insert***, ***update***, ***delete***), позволяющая реализовать более сложные проверки, чем это возможно с помощью ограничений на значения данных и ссылочной целостности. Триггер всегда относится к одной таблице и к одной или нескольким из указанных операций.

Триггеры могут быть двух уровней:

(i) уровня строки[1] (***for each row***) — запускаются для каждой строки таблицы, затронутой изменением;

(ii) уровня оператора — запускаются для каждой из инструкций ***insert***, ***update***, ***delete***, применяемой к таблице.

Триггеры могут быть одного из трех видов:

(i) ***after*** — триггер запускается после внесения изменений в таблицу или строку (для триггеров ***for each row***);

(ii) ***before*** — триггер запускается до внесения изменений в строку (для триггеров ***for each row***)[2];

(iii) ***instead of*** — триггер запускается вместо внесения изменений в таблицу или строку (для триггеров ***for each row***).

У таблицы может быть только один триггер ***instead of*** для каждой из операций модификации данных.

В течение выполнения триггера существуют две таблицы ***inserted*** и ***deleted***. Таблица ***inserted*** содержит новые версии строк (вставленные оператором ***insert*** или измененные оператором ***update***). Таблица ***deleted*** содержит старые версии строк (удаленные оператором ***delete*** или подлежащие изменению оператором ***update***). Ссылка на указанные таблицы производится так же, как на основные таблицы БД.

7.2 Создание триггеров

Триггер создается оператором языка SQL, имеющим следующий синтаксис [CREATE TRIGGER (Transact-SQL) - SQL Server | Microsoft Docs]:

```
create trigger <имя триггера>
on <имя таблицы>
{ for | after | instead of } { [ insert ] [ , ] [ update ] [ , ] [ delete ] }
as
        <оператор>
```

[1] В MS SQL не поддерживаются, поддерживаются, например, в SQLA [CREATE TRIGGER Statement - SAP Help Portal].
[2] В MS SQL не поддерживаются, поддерживаются, например, в SQLA.

Если при создании триггера использована директива ***for***, ***after*** действует по умолчанию, т. е. триггер запускается после внесения изменений в таблицу, в том числе после проверки ограничений на значения данных и ссылочной целостности, выполнения каскадных операций. При создании триггера должна быть указана хотя бы одна из операций, при выполнении которой он запускается.

Для триггеров ***instead of*** нельзя использовать ***delete***, если для таблицы задана обработка ссылочной целостности с помощью ***on delete***. Аналогичное ограничение действует для ***update***. Если для таблицы заданы ограничения ссылочной целостности, их проверка осуществляется до запуска триггера ***after***, но после срабатывания триггера ***instead of***. В случае возникновения ошибки ссылочной целостности триггер ***after*** не запускается, а для триггера ***instead of*** выполняется откат изменений. Если триггер ***instead of*** выполняет операцию, которая должна приводить к срабатыванию триггера ***instead of***, MSSQL блокирует рекурсивный запуск триггера.

MS SQL допускает создание для одной таблицы нескольких триггеров вида ***after***. С помощью СХП ***sp_settriggerorder*** можно указать, какой из триггеров будет запускаться первым, а какой последним, остальные триггеры будут стартовать случайным образом [Указание первого и последнего триггеров - SQL Server | Microsoft Docs]. Действия, выполняемые в теле триггера, могут приводить к срабатыванию другого (вложенные триггеры [Создание вложенных триггеров - SQL Server | Microsoft Docs]) или этого же триггера (рекурсия), разрешение или запрет вложенных триггеров осуществляется установкой параметра сервера ***nested triggers*** посредством вызова СХП ***sp_configure***. Возможность рекурсивного срабатывания триггеров регулируется параметром базы данных ***RECURSIVE_TRIGGERS***, задаваемым с помощью директивы ***alter database … set*** (прямая рекурсия) или параметром сервера ***nested triggers*** (косвенная рекурсия).

Переопределение триггера осуществляет оператор ***alter trigger***, синтаксис которого аналогичен ***create trigger***, для удаления триггера служит ***drop trigger***

<имя триггера>. СХП ***sp_helptrigger <имя таблицы>*** выводит информацию о триггерах, а ***sp_helptext <имя триггера>*** возвращает определение триггера. Работу конкретных или всех триггеров для таблицы можно приостановить и возобновить директивой ***alter table <имя таблицы> { enable | disable } trigger { all | <имя триггера> [, …] }*** или операторами ***enable trigger*** и ***disable trigger***.

Примеры триггеров:

(i) ограничение численности студентов в группе:

```
create trigger Вст_Студент
on Студент
instead of insert
as
   if exists (select Группа, count(*)    from Студент
                     where Группа in (select Группа from inserted)
                     group by Группа
                     having count(Ном_Зач) >= 20)
        print 'Слишком много студентов в группе'
   else
        insert into Студент select * from inserted
go
```

(ii) удаление группы, если в ней больше нет студентов:

```
create trigger Уд_Студент
on Студент
for delete
as
   delete Группа
        where        Id in (select Группа from deleted) and not exists
                (select * from Студент where Группа = Id )
go
```

(iii) контроль перерасхода стипендиального фонда:

```
create trigger Изм_Студент
on Студент
for update
as
   if (select sum(Стипендия) from Студент) > 1000000
   begin
        print 'Стипендиальный фонд превышен'
        rollback transaction
```

```
    end
go
```

(iv) формирование статистики:

— создадим в БД таблицу ***Успеваемость_345***, которая будет содержать количество оценок отл., хор., уд., полученных студентами группы по дисциплине:

```
create table Успеваемость_345 (
  Id_Группы smallint not null,
  Id_Дисциплины smallint not null,
  Количество3 int,
  Количество4 int,
  Количество5 int
  primary key (Id_Группы, Id_Дисциплины))
go
```

— предполагая, что приложение, работающее с базой данных, всегда будет модифицировать одну запись, создадим триггер, который будет изменять данные в этой таблице при добавлении, изменении, удалении данных в таблице ***Успеваемость***:

```
create trigger Изм_Успеваемость
on Успеваемость
for insert, update, delete
as
        declare @ci int, @cd int
        select @ci = count(*) from inserted
        select @cd = count(*) from deleted
        if @ci > 1 or @cd > 1 return -- инструкция затронула > 1 строки

        declare @Id_Группы smallint, @Id_Дисциплины smallint
        declare @Оценка_i tinyint, @Оценка_d tinyint

        if @ci = 1 begin
                select @Id_Группы = Студент.Группа from inserted,
        Студент
                where inserted.Ном_зач = Студент.Ном_зач
                select @Id_Дисциплины = Id_Дисциплины,
                @Оценка_i = Оценка from inserted
        end

        if @cd = 1 begin
                select @Id_Группы = Студент.Группа from deleted,
```

```
Студент
        where deleted.Ном_зач = Студент.Ном_зач
        select @Id_Дисциплины = Id_Дисциплины,
        @Оценка_d = Оценка from deleted
end

if @ci = 1
        if not exists (select * from  Успеваемость_345
        where          @Id_Группы = Id_Группы and
                @Id_Дисциплины = Id_Дисциплины)
                insert into Успеваемость_345
                values (@Id_Группы, @Id_Дисциплины, 0, 0, 0)

if @ci = 1 begin
        if @Оценка_i = 3
                update Успеваемость_345
                set Количество3 = Количество3 + 1
                where          @Id_Группы = Id_Группы and
                        @Id_Дисциплины = Id_Дисциплины
        if @Оценка_i = 4
                update Успеваемость_345
                set Количество4 = Количество4 + 1
                where          @Id_Группы = Id_Группы and
                        @Id_Дисциплины = Id_Дисциплины
        if @Оценка_i = 5
                update Успеваемость_345
                set Количество5 = Количество5 + 1
                where          @Id_Группы = Id_Группы and
                        @Id_Дисциплины = Id_Дисциплины
end

if @cd = 1 begin
        if @Оценка_d = 3
                update Успеваемость_345
                set Количество3 = Количество3 - 1
                where          @Id_Группы = Id_Группы and
                        @Id_Дисциплины = Id_Дисциплины
        if @Оценка_d = 4
                update Успеваемость_345
                set Количество4 = Количество4 - 1
                where          @Id_Группы = Id_Группы and
                        @Id_Дисциплины = Id_Дисциплины
        if @Оценка_d = 5
                update Успеваемость_345
```

```
                    set Количество5 = Количество5 - 1
                    where        @Id_Группы = Id_Группы and
                    @Id_Дисциплины = Id_Дисциплины

            end
        go
```

Переменные ***@ci*** и ***@cd*** получают значения числа строк в таблицах ***inserted*** и ***deleted***. Если число строк, затронутых оператором модификации данных, превышает одну, осуществляется выход из триггера. Далее из таблицы ***inserted*** (***@ci = 1***) и/или ***deleted*** (***@cd = 1***) выбираются идентификаторы группы и дисциплины, переменные ***@Оценка_i*** и ***@Оценка_d*** получают новое (триггер запустился при выполнении ***insert*** или ***update***) и старое (триггер запустился при выполнении ***delete*** или ***update***) значение оценки. Если записи с такими идентификаторами нет в таблице ***Успеваемость_345***, производится ее вставка. Далее в зависимости от значений в ***@Оценка_i*** и ***@Оценка_d*** увеличиваются или уменьшаются значения счетчиков в таблице ***Успеваемость_345***.

Заметим, что инструкции вида:

```
insert Успеваемость (Ном_зач, Оценка, Id_Дисциплины)
values (87, 3, 1), (87, 4, 2), (87, 3, 3)
go
```

а также операторы ***update*** и ***delete***, затрагивающие несколько строк, не будут приводить к каким-либо изменениям в таблице ***Успеваемость_345***.

Для определения числа строк, затронутых последним выполненным оператором модификации данных, можно использовать глобальную системную переменную ***@@rowcount***.

Рассмотренные триггеры относятся к классу DML, в них не разрешены инструкции:

— ***{ alter | create | drop } { database | index }***;

— ***alter table***, затрагивающие столбцы, первичный ключ и ***unique***.

7.3 Лабораторная работа 7

По аналогии с примерами, приведенными в п. 7.2, создать:

— триггеры каждого вида (***after***, ***instead of***) для каждой из операций (***insert***, ***update***, ***delete***), обеспечивающие активную целостность данных;

— триггер, реализующий вычисления или формирование статистики или ведение истории изменений в БД.

Содержание отчета:

— тексты триггеров (с пояснениями/комментариями);

— SQL операторы, вызывающие запуск триггеров, и наборы данных, иллюстрирующие их работу.

Варианты заданий приведены в ПРИЛОЖЕНИИ.

8. КУРСОРЫ

8.1 Назначение курсоров

При выполнении некоторых видов вычислений и представлении данных в интерфейсе пользователя бывает необходимо организовывать последовательный просмотр и обработку записей в наборе данных. Реализацию данных механизмов в языке SQL обеспечивают курсоры. Типовая последовательность работы с курсором включает следующие операции:

(i) объявление курсора оператором ***declare <имя курсора> [scroll] cursor for <оператор select> [for { read only | update [of <список столбцов >] }]*** ; директива ***scroll*** указывается для двунаправленного курсора; директива ***read only*** предотвращает выполнение изменений посредством курсора; директива ***update*** задает столбцы, изменение которых возможно через курсор, если ***<список столбцов >*** опущен, допускается изменение всех столбцов;

(ii) открытие курсора оператором ***open <имя курсора>***; при открытии формируется набор данных, возвращаемый курсором;

(iii) перемещение по записям, возвращаемым курсором, с помощью оператора ***fetch [{ next | prior | first | last | absolute <номер строки от начала курсора> | relative <номер строки относительно текущей +/-> }] from <имя курсора> into [<имя переменной> [, …]]*** , и выполнение для них необходимой обработки; в однонаправленных курсорах (директива ***scroll*** была опущена при создании курсора) допускается указание только параметра ***next***;

(iv) закрытие курсора оператором ***close <имя курсора>***; при закрытии курсора происходит высвобождение результирующего набора данных, сформированного курсором, закрытый курсор впоследствии может быть повторно открыт;

(v) освобождение курсора оператором ***deallocate <имя курсора>***; освобождение курсора высвобождает структуры данных, используемые курсором.

После выполнения ***fetch*** переменная @@***fetch_status*** получает значение 0 при успешном выполнении, -1 если выполнение неудачно или строка вне набора данных, -2 если выбранная строка отсутствует.

Пусть в БД существует таблица ***Успеваемость_345***, созданная в предыдущем разделе. Удалим в ней все записи, вставим информацию о группах и дисциплинах, у которых есть оценки, и выполним посредством курсора вычисление значений столбцов ***Количество3***, ***Количество4***, ***Количество5***:

```
create proc Вычисл_Успеваемость_345
as
        delete Успеваемость_345

        insert into Успеваемость_345 (Id_Группы, Id_Дисциплины, Количество3,
                                      Количество4, Количество5)
        select  distinct Студент.Группа, Успеваемость.Id_Дисциплины, 0, 0, 0
        from Студент, Успеваемость
        where Студент.Ном_зач = Успеваемость.Ном_зач

        declare @Группа smallint, @Дисциплина smallint, @Оценка tinyint,
                @Количество int

        declare Вычисл_345 cursor for
        select Группа, Id_Дисциплины, Оценка, count(Id) 'Количество'
        from Студент, Успеваемость
```

```
        where Студент.Ном_зач = Успеваемость.Ном_зач
        group by Группа, Id_Дисциплины, Оценка
        open Вычисл_345

        fetch next from Вычисл_345 into      @Группа, @Дисциплина,
                                             @Оценка, @Количество
        while @@fetch_status = 0 begin
                if @Оценка = 3
                        update Успеваемость_345
                        set Количество3 = @Количество
                        where        @Группа = Id_Группы and
                                     @Дисциплина = Id_Дисциплины
                if @Оценка = 4
                        update Успеваемость_345
                        set Количество4 = @Количество
                        where        @Группа = Id_Группы and
                                     @Дисциплина = Id_Дисциплины
                if @Оценка = 5
                        update Успеваемость_345
                        set Количество5 = @Количество
                        where        @Группа = Id_Группы and
                                     @Дисциплина = Id_Дисциплины
                fetch next from Вычисл_345 into      @Группа, @Дисциплина,
                                                     @Оценка, @Количество
        end

        close Вычисл_345
        deallocate Вычисл_345
go
exec Вычисл_Успеваемость_345
go
```

Запрос, на котором основан курсор, возвращает значения групповой агрегатной функции ***count*** для каждой студенческой группы, дисциплины и значения оценки. Цикл ***while*** просматривает данный набор и в зависимости от значения оценки модифицирует один из столбцов ***Количество3***, ***Количество4***, ***Количество5***.

В предыдущем разделе был разработан триггер ***Изм_Успеваемость***, модифицирующий данные в таблице ***Успеваемость_345*** при изменении строки в

таблице ***Успеваемость***. Создадим триггер, который посредством курсоров будет обрабатывать несколько записей в таблицах ***inserted*** и ***deleted***:

```
create trigger Изм_Успеваемость_Курс
on Успеваемость
for insert, update, delete
as
	declare @ci int, @cd int
	select @ci = count(*) from inserted
	select @cd = count(*) from deleted
	if @ci <= 1 and @cd <= 1 return -- инструкция затронула 1 строку
	                                -- вычисления выполнит Изм_Успеваемость
	declare @Группа smallint, @Дисциплина smallint, @Оценка tinyint,
		@Ном_зач int

	declare Вычисл_345_i cursor for
		select Ном_зач, Id_Дисциплины, Оценка
		from inserted
	  open Вычисл_345_i

	  fetch next from Вычисл_345_i into	@Ном_зач, @Дисциплина, @Оценка
	  while @@fetch_status = 0 begin
		select @Группа = Группа from Студент where Ном_зач = @Ном_зач
		if not exists (select * from Успеваемость_345
				where	Id_Группы = @Группа and
					Id_Дисциплины = @Дисциплина)
			insert into Успеваемость_345 (Id_Группы, Id_Дисциплины,
				Количество3, Количество4, Количество5)
			values (@Группа, @Дисциплина, 0, 0, 0)
		if @Оценка = 3
			update Успеваемость_345
			set Количество3 = Количество3 + 1
			where	@Группа = Id_Группы and
				@Дисциплина = Id_Дисциплины
		if @Оценка = 4
			update Успеваемость_345
			set Количество4 = Количество4 + 1
			where	@Группа = Id_Группы and
				@Дисциплина = Id_Дисциплины
		if @Оценка = 5
			update Успеваемость_345
			set Количество5 = Количество5 + 1
			where	@Группа = Id_Группы and
				@Дисциплина = Id_Дисциплины
```

```
        fetch next from Вычисл_345_i into   @Ном_зач,        @Дисциплина,
                @Оценка
end

close Вычисл_345_i
deallocate Вычисл_345_i

declare Вычисл_345_d cursor for
select Ном_зач, Id_Дисциплины, Оценка from deleted
open Вычисл_345_d

fetch next from Вычисл_345_d into @Ном_зач, @Дисциплина, @Оценка
while @@fetch_status = 0 begin
        select @Группа = Группа from Студент where Ном_зач = @Ном_зач
        if @Оценка = 3
                update Успеваемость_345
                set Количество3 = Количество3 - 1
                where           @Группа = Id_Группы and
                                @Дисциплина = Id_Дисциплины
        if @Оценка = 4
                update Успеваемость_345
                set Количество4 = Количество4 - 1
                where           @Группа = Id_Группы and
                                @Дисциплина = Id_Дисциплины
        if @Оценка = 5
                update Успеваемость_345
                set Количество5 = Количество5 - 1
                where           @Группа = Id_Группы and
                                @Дисциплина = Id_Дисциплины
        fetch next from Вычисл_345_d into @Ном_зач, @Дисциплина,
                @Оценка
end

close Вычисл_345_d
deallocate Вычисл_345_d
go
```

Здесь первый ***while*** обеспечивает просмотр таблицы ***inserted*** посредством курсора ***Вычисл_345_i***, а второй – таблицы ***deleted*** посредством курсора ***Вычисл_345_d***. В случае если в ***inserted*** первая оценка по данной дисциплине в данной группе, в ***Успеваемость_345*** осуществляется добавление новой записи.

Данный триггер будет корректно обрабатывать инструкции вида:

```
insert Успеваемость (Ном_зач, Оценка, Id_Дисциплины)
values (87, 3, 1), (87, 4, 2), (87, 3, 3)
go
```

а также операторы ***update*** и ***delete***, затрагивающие несколько строк.

8.2 Виды курсоров

Синтаксис оператора ***declare***, приведенный в п. 8.1 соответствует стандарту ISO, диалект Transact SQL включает также альтернативный оператор ***declare***, учитывающий расширенные относительно стандарта возможности курсоров в MS SQL: ***declare <имя курсора> cursor [{ local | global }] [{ forward only | scroll }] [{ static | keyset | dynamic | fast_forward }] [{ read_only | scroll_locks | optimistic }] for <оператор select> [for update [of <список столбцов >]]*** .

Для локальных курсоров (***local***) область действия определения курсора ограничена ПЗ, ХП, Т, где он объявлен, локальные курсоры неявно освобождаются по завершении ПЗ, ХП, Т. Областью действия глобального курсора (***global***) является сеанс работы с базой данных, как следствие, он доступен из любых ПЗ или ХП, выполняемых в данном сеансе. Неявное освобождение глобального курсора происходит при разрыве соединения.

Для однонаправленных курсоров (***forward only***) в операторе ***fetch*** допустима только директива ***next***. Если в определении курсора не указывается ни ***{ forward only | scroll }***, ни ***{ static | keyset | dynamic }***, по умолчанию будет действовать ***forward only***. Если ***{ static | keyset | dynamic }*** задается, по умолчанию действует ***scroll***.

Статические курсоры (***static***) реализуются посредством создания временной таблицы в ***tempdb***, в которую копируются данные, изменения данных в базовых таблицах не влияют на курсор, курсор не позволяет производить изменение данных в них.

В курсорах ***keyset*** множество и порядок строк, возвращаемых курсором, не изменяется в части ключевых полей, изменения неключевых полей сделанные в

базовых таблицах отображаются курсором. Если в базовой таблице выполняется удаление строки, присутствующей в курсоре, при перемещении на нее указателя курсора @@***fetch_status*** получит значение -2. Строки, вставленные в базовую таблицу, удовлетворяющие условию запроса, на котором основан курсор, курсором не отображаются. Изменение ключа записи в базовой таблице, приводящее к «появлению» или «исчезновению» записи из курсора обрабатываются аналогично.

Динамические курсоры (***dynamic***) отображают все изменения, происходящие в базовых таблицах, для них не допустима директива ***absolute*** в операторе ***fetch***.

Курсоры, для которых включена оптимизация производительности (***fast_forward***), относятся к виду ***read_only***, для них не допускаются директивы ***scroll*** и ***for update***.

Курсоры только для чтения (***read_only***) запрещают изменение данных через курсор, ***операторы*** update и ***delete*** не могут содержать условия ***where current of***, ссылающиеся на данный курсор. Параметры ***scroll_locks*** и ***optimistic*** задают режимы пессимистичной и оптимистичной блокировки строк при работе с курсором.

Создадим динамический курсор, выводящий записи из таблицы ***Успеваемость_345*** с задержкой 3 сек.

```
declare Вывод_345 cursor dynamic for
        select Id_Группы, Id_Дисциплины, Количество3, Количество4,
                                Количество5
        from Успеваемость_345
open Вывод_345

declare  @Id_Группы smallint, @Id_Дисциплины smallint,
                @Количество3 int, @Количество4 int,  @Количество5 int,
                        @Группа char(7), @Дисциплина varchar(50)

fetch next from Вывод_345 into @Id_Группы, @Id_Дисциплины,
        @Количество3, @Количество4,  @Количество5
waitfor delay '00:00:03'
while @@fetch_status = 0 begin
```

```
        select @Группа = Номер from Группа where Id = @Id_Группы
        select @Дисциплина = Наименование from Дисциплина
        where Id = @Id_Дисциплины

        print @Группа + '  ' + @Дисциплина + '  ' +
                convert(varchar(10), @Количество3) + '  ' +
                convert(varchar(10), @Количество4) + '  ' +
                convert(varchar(10), @Количество5)

        fetch next from Вывод_345 into @Id_Группы, @Id_Дисциплины,
                @Количество3, @Количество4,  @Количество5
        end

close Вывод_345
deallocate Вывод_345
go
```

Если во время выполнения данного ПЗ из другого сеанса (другого окна запроса) произвести вставку записи в таблицу ***Успеваемость***, в результате срабатывания триггера ***Изм_Успеваемость*** или ***Изм_Успеваемость_Курс*** набор данных, возвращаемый курсором изменится. При изменении вида курсора на ***static***, этого не происходит.

MS SQL может создать курсор, отличающийся от вида, запрашиваемого в ***declare*** если запрос, на котором основывается курсор конфликтует с функциями курсора запрашиваемого типа. Получить атрибуты курсора можно с помощью СХП ***sp_describe_cursor***.

Функция ***@@cursor_rows*** возвращает число строк в открытом курсоре, для динамических курсоров возвращается значение -1.

8.3 Лабораторная работа 8

По аналогии с примерами, приведенными в п. 8.1, 8.2:

— реализовать ПЗ или ХП или Т, использующие курсоры (ISO Syntax);

—продемонстрировать различия в работе статических и динамических курсоров, а также курсоров ***keyset*** (Transact-SQL Extended Syntax);

— используя директивы ***update*** и ***delete*** с условиями ***where current of <имя курсора>*** самостоятельно реализовать примеры изменения и удаления записей посредством курсора.

Содержание отчета:

— тексты ПЗ, ХП, Т (с пояснениями/комментариями);

— наборы данных, формируемые при выполнении ПЗ, ХП, Т.

Варианты заданий приведены в ПРИЛОЖЕНИИ.

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. Г. Гарсиа-Молина Г., Ульман Дж. Д., Д. Уидом Д. Системы баз данных. Полный курс./Пер. с англ. — М.: Вильямс, 2003. — 1088 с.
2. Коннолли Т, Бегг К. Базы данных. Проектирование, реализация и сопровождение. Теория и практика./Пер. с англ. —3-е изд. — М.: Вильямс, 2003. — 1436 с.
3. Дейт К. Дж. Введение в системы баз данных./Пер. с англ. — 8-е изд. — М.: Вильямс, 2005. — 1138 с.
4. Роб П. Системы баз данных: проектирование, реализация и управление. — 5-е изд. — СПб.: БХВ - Петербург, 2004. — 1040 с.
5. Мейер Д. Теория реляционных баз данных. /Пер. с англ. — М.: Мир, 1987. — 608 с.
6. Цикритзис Д, Лоховски Ф. Модели данных./Пер. с англ.— М.: Финансы и статистика, 1985.. — 344 с.
7. Ульман Дж. Д., Д. Уидом Д. Введение в системы баз данных. /Пер. с англ. — М.: Лори, 2000. — 374 с.

СОДЕРЖАНИЕ

1. Создание таблиц базы данных ____ *3*

1.1 Базы данных и СУБД ____ **3**

1.2 Таблицы ____ **5**

1.3 Ссылочная целостность ____ **12**

1.4 Лабораторная работа 1 ____ **13**

2. Заполнение и модификация таблиц базы данных ____ *14*

2.1 Вставка данных в таблицы ____ **14**

2.2 Изменение данных ____ **16**

2.3 Удаление данных ____ **17**

2.4 Изменение определения таблицы ____ **18**

2.5 Лабораторная работа 2 ____ **22**

3. Запросы на языке SQL: выборка данных ____ *23*

3.1 Оператор select ____ **23**

3.2 Директивы, используемые в условиях запросов ____ **28**

3.3 Лабораторная работа 3 ____ **29**

4. Запросы на языке SQL: агрегатные функции ____ *30*

4.1 Агрегатные функции ____ **30**

4.2 Объединение, пересечение, разность запросов ____ **33**

4.3 Лабораторная работа 4 ____ **34**

5. Запросы на языке SQL: подзапросы ____ *35*

5.1 Запросы с подзапросам ____ **35**

5.2 Производные таблицы и представления ____ **37**

5.3 Экзистенциальные запросы ____ **39**

5.4 Лабораторная работа 5 42

6. Хранимые процедуры и функции 43

6.1 Управляющие конструкции в языке SQL 43

6.2 Хранимые процедуры 48

6.3 Хранимые функции 52

6.4 Лабораторная работа 6 53

7. Триггеры 54

7.1 Виды триггеров 54

7.2 Создание триггеров 55

7.3 Лабораторная работа 7 61

8. Курсоры 61

8.1 Назначение курсоров 61

8.2 Виды курсоров 66

8.3 Лабораторная работа 8 68

Библиографический список 69

Содержание 70

ПРИЛОЖЕНИЕ 72

Варианты заданий 72

ПРИЛОЖЕНИЕ

Варианты заданий

1. Создайте базу данных для хранения следующих сведений: алфавитный и тематический каталоги книг в библиотеке, читатель, читательский билет, формуляр читателя (выданные и возвращенные книги), напоминание о необходимости возврата книги. Составьте запросы, позволяющие выбрать:
   а) читателей, которые брали книги на прошлой неделе;
   б) читателей, которые брали книги, написанные Ахо и Ульманом (в соавторстве);
   в) читателей, у которых на руках две или более книги одного автора;
   г) количество книг, находящихся на руках у каждого из читателей;
   д) читателей, прочитавших более ста книг;
   е) читателей, у которых на руках максимальное количество книг;
   ж) читателей, взявших книги, которые больше никому не выдавались;
   з) читателей, читающих книги всех жанров;
   и) читателей, которые читают только книги жанра «приключения».
2. Создайте базу данных для хранения следующих сведений: специальность, учебный план, кафедра, преподаватель, дисциплина, группа, курс, вид занятия, пара. Составьте запросы, позволяющие выбрать:
   а) преподавателей, ведущих Базы данных на различных факультетах;
   б) преподавателей, ведущих как Базы данных, так и Логическое программирование;
   в) преподавателей, которые ведут более двух видов занятий по одной дисциплине;
   г) количество дисциплин для каждого преподавателя;
   д) группы, у которых в среднем менее 3-х пар в день;
   е) преподавателей, ведущих более трех различных дисциплин;

ж) преподавателей, которые ведут занятия только на старших курсах;

з) преподавателей, ведущих все виды занятий;

и) преподавателей, ведущих занятия в максимальном количестве групп.

3. Создайте базу данных для хранения следующих сведений: турфирма, тур, страна, турист, путевка, загранпаспорт, виза, ваучер отеля, билет. Составьте запросы, позволяющие выбрать:

   а) туристов, посещавших в прошлом году Италию и Францию;

   б) турфирмы, продающие туры в Египет и Турцию;

   в) туристов, пользовавшихся услугами двух и более турфирм;

   г) количество путевок, проданных каждой из турфирм за прошлый год;

   д) среднюю цену путевки в Тунис;

   е) туристов, побывавших во Франции более пяти раз;

   ж) туристов, побывавших только в одной стране;

   з) туристов, побывавших во всех странах, в которые предлагаются туры;

   и) пары туристов, которые всегда путешествуют вместе.

4. Создайте базу данных для хранения следующих сведений: студент, группа, дисциплина, преподаватель, лабораторная работа, рейтинг за сданную лабораторную работу. Составьте запросы, позволяющие выбрать:

   а) максимальный рейтинг, который может получить студент за работу №8 по БД;

   б) работы и рейтинги, сданные и полученные конкретным студентом;

   в) дисциплины, у которых есть лабораторные работы с одинаковыми названиями;

   г) количество работ, сданных каждым студентом по БД;

д) студентов, у которых средний рейтинг за сданные лабораторные работы по БД превышает 4;
е) студентов, не сдавших ни одной работы по БД;
ж) лабораторные по БД, которые нужно досдать Сыроежкину из группы 4000;
з) студентов, получивших одинаковый рейтинг за все работы по БД;
и) студентов, сдавших все работы по БД.

5. Создайте базу данных для хранения следующих сведений: ВУЗ, студент, группа, факультет, конференция, тема доклада, программа конференции. Составьте запросы, позволяющие выбрать:
   а) студентов первого факультета, выступавших на конференции Информатика;
   б) темы докладов студентов для заданной группы;
   в) выступления, подготовленные двумя студентами различных факультетов;
   г) количество докладов для каждой конференции;
   д) среднее количество докладов, сделанных студентами третьего факультета на конференциях;
   е) студентов, выступивших на трех или большем числе конференций;
   ж) студентов четвертого факультета, не выступавших на конференциях;
   з) студентов, выступивших на всех конференциях;
   и) пары студентов, всегда выступающие вместе.
6. Создайте базу данных для хранения следующих сведений: компьютерный магазин, модель компьютера, комплектующие, производители, поставщики, поставка, заказ, сборка. Составьте запросы, позволяющие выбрать:
   а) модели компьютеров, в которых используются винчестеры Samsung;

б) модели компьютеров, в которых используются как накопители SSD, так и HDD;
в) модели компьютеров, имеющие одинаковый размер оперативной и внешней памяти;
г) количество моделей, продаваемых в каждом из магазинов;
д) магазины, в которых средняя цена компьютера ниже, чем в других;
е) магазины, в которых продается наибольшее количество моделей;
ж) модели компьютеров, не имеющие накопителей DVD;
з) магазины, в которых не продаются модели, укомплектованные одновременно оборудованием Intel и Samsung;
и) модели компьютеров, укомплектованные всеми типами периферийных устройств.

7. Создайте базу данных для хранения следующих сведений: номер маршрута автобуса, остановка, транспортная компания, лицензия, график движения, автобус, водитель. Составьте запросы, позволяющие выбрать:
   а) маршруты, выполняемые заданной компанией;
   б) маршруты, которыми можно доехать до Дворцовой площади;
   в) маршруты, имеющие общие остановки;
   г) количество маршрутов, обслуживаемых каждой компанией;
   д) компании, обслуживающие наибольшее число маршрутов;
   е) компании, средняя продолжительность маршрутов которых ниже чем у других;
   ж) компании, маршруты которых не останавливаются на Дворцовой площади;
   з) компании, у которых нет маршрутов короче, чем 10 остановок;
   и) маршруты, которые включают все остановки заданного маршрута.
8. Создайте базу данных для хранения следующих сведений: театр, спектакль, жанр, автор, режиссер, актер, репертуар. Составьте запросы, позволяющие выбрать:

а) спектакли жанра комедия;
б) спектакли, в которых занят заданный актер;
в) спектакли, идущие более чем в одном театре;
г) количество спектаклей для каждого из театров;
д) театры, в которых количество драм превышает число комедий;
е) спектакли, в которых занято наибольшее число актеров;
ж) спектакли одного актера;
з) театры, в которых идут спектакли всех жанров;
и) актеров, занятых только в одном театре.

9. Создайте базу данных для хранения следующих сведений: аптека, медикамент: международное непатентованное и торговое наименование, состав, дозировка, упаковка, форма выпуска, цена, производитель. Составьте запросы, позволяющие выбрать:
   а) аптеки, в которых есть лекарства заданного производителя;
   б) аптеки, в которых продается одно и то же лекарство различных производителей;
   в) цена аспирина в различных аптеках;
   г) количество наименований лекарств, продающихся в каждой из аптек;
   д) аптеки, в которых цена аспирина минимальна;
   е) средняя стоимость аспирина компании АБВ в аптеках;
   ж) аптеки, в которых нет медикаментов, заданного производителя;
   з) пары производителей, у которых нет ни одного одинакового медикамента;
   и) аптеки, в которых есть все лекарства.
10. Создайте базу данных для хранения следующих сведений: фильм, сериал, литературная основа, студия, жанр, актер, режиссер, сценарист, продюсер. Составьте запросы, позволяющие выбрать:
   а) список фильмов, снятых заданной студией за заданный период;

б) перечень студий, в фильмах которых играл заданный актер;
в) актеров, снимавшихся как в комедиях, так и в мелодрамах;
г) студии, на которых количество мелодрам превышает число комедий;
д) актеров, снявшихся в десяти фильмах;
е) среднее количество фильмов каждого из жанров, снимающееся на студии за год;
ж) студии, на которых снимаются фильмы только одного жанра;
з) студии, на которых снимаются фильмы всех жанров;
и) студии, никогда не выпускавшие ремейков.

11. Создайте базу данных для хранения следующих сведений: автомобиль, модель, производитель, дилер, город, цена, продажа, владелец, техническое обслуживание. Составьте запросы, позволяющие выбрать:
    а) перечень моделей для заданного дилера;
    б) дилеров, представляющих одновременно Toyota и УАЗ;
    в) дилеров, продающих автомобили иностранного производства;
    г) количество автомобилей, проданных каждым из дилеров в прошлом году;
    д) среднюю цену Toyota Camry у дилеров в Москве;
    е) производителей, у которых наибольшее количество дилеров в Санкт-Петербурге;
    ж) дилеров, не продающих одновременно Ford и Renault;
    з) дилеров, предлагающих модели всех производителей;
    и) дилеров, у которых нет моделей дороже 300000.

12. Создайте базу данных для хранения следующих сведений: подразделение, штатное расписание, должность, сотрудник, дети, прием, перевод, увольнение. Составьте запросы, позволяющие выбрать:
    а) список сотрудников заданного подразделения;
    б) подразделения, входящие в состав заданного;

в) сотрудников, у которых есть дети различного пола;

г) среднюю численность подразделений;

д) инженеров, у которых более пяти детей;

е) подразделения, в которых количество техников превышает количество инженеров;

ж) подразделения, в которых не работают совместители;

з) подразделения, в которых представлены все должности;

и) сотрудников, у которых все дети одного пола.

13. Создайте базу данных для хранения следующих сведений: издательство, автор, книга, жанр, план издания, тираж, реализация. Составьте запросы, позволяющие выбрать:

а) перечень книг, выпущенных заданным издательством в прошлом году;

б) авторы, сотрудничающие с несколькими издательствами;

в) книги, написанные в соавторстве;

г) количество книг каждого жанра, выпущенных каждым издательством;

д) авторов, написавших наибольшее количество книг;

е) средний объем книг, выпускаемых заданным издательством;

ж) издательства, выпускающие только сказки и детективы;

з) издательства, выпускающие книги всех жанров;

и) издательства, не выпустившие ни одной книги в 2019 году.

14. Создайте базу данных для хранения следующих сведений: врач, специальность, пациент, прием, история болезни (медицинская карта). Составьте запросы, позволяющие выбрать:

а) список пациентов, принятых терапевтами вчера;

б) врачей, совмещающих различные специальности;

в) пациентов, посещавших и хирурга, и кардиолога;

г) количество пациентов, принятых каждым из врачей за прошедший год;
д) врачи, принявшие меньше всего пациентов;
е) врачей, у которых количество принимаемых пациентов превышает среднее;
ж) пациентов, которые никогда не посещали хирурга;
з) пациентов, которые посетили всех специалистов;
и) врачи, не совмещающие работу по различным специальностям.

15. Создайте базу данных для хранения следующих сведений: авиакомпания, авиарейс, расписание, самолет, экипаж, аэропорт, количество и длина полос. Составьте запросы, позволяющие выбрать:
    а) список рейсов для заданной авиакомпании;
    б) типы самолетов, используемые заданной авиакомпанией;
    в) авиакомпании, у которых прямой и обратный рейс выполняют различные типы самолетов;
    г) направления, на которых работает более трех авиакомпаний;
    д) количество авиарейсов, выполняемых между каждой парой аэропортов;
    е) авиакомпании, выполняющие максимальное количество рейсов;
    ж) авиакомпании, не работающие в Стамбуле;
    з) авиакомпании, использующие все типы самолетов;
    и) авиакомпании, у которых все самолеты одного производителя.

16. Создайте базу данных для хранения следующих сведений: город, район, квартира, комната, площадь, цена, агент, продажа. Составьте запросы, позволяющие выбрать:
    а) перечень однокомнатных квартир, продаваемых в Московском районе;
    б) квартиры, находящиеся на одной улице, но в различных районах;
    в) двух- и трехкомнатные квартиры, имеющие одинаковую площадь;

г) средняя цена однокомнатной квартиры в городе;
д) районы, в которых продается наибольшее число объектов недвижимости;
е) районы, в которых минимальна стоимость квадратного метра;
ж) улицы, продолжительность которых ограничивается только одним районом;
з) районы, в которых не продаются однокомнатные квартиры;
и) районы, в которых продаются квартиры всех строительных серий.

17. Создайте базу данных для хранения следующих сведений: олимпиада, страна, спортсмен, команда, вид спорта, место. Составьте запросы, позволяющие выбрать:
    а) страны, принявшие участие в зимней олимпиаде 2018 г.;
    б) спортсменов, принявших участие как в летних, так и в зимних олимпиадах;
    в) спортсменов, получивших золото по двум или более видам соревнований на одной и той же олимпиаде;
    г) среднее число спортсменов, выставляемых каждой страной на каждый из видов олимпиад;
    д) страны, количество побед которых на зимних олимпиадах, превышает количество побед на летних;
    е) страны, завоевавшие наибольшее количество наград в 2018 г.;
    ж) страны, никогда не участвовавшие в зимних олимпиадах;
    з) страны, не участвовавшие в олимпиадах в период 2008…2018 г. г.;
    и) страны, завоевавшие призовые места по всем видам спорта.

18. Создайте базу данных для хранения следующих сведений: город, ж/д станция, поезд, остановки, вагон, место, пассажир, билет. Составьте запросы, позволяющие выбрать:
    а) пассажиров, покупавших билеты в прошлом месяце из СПб в Москву;

б) пассажиров, покупавших в течение месяца и прямые и обратные билеты;
в) поезда, в которых есть и купейные и плацкартные и сидячие вагоны;
г) количество поездов из СПб в Москву;
д) количество билетов, проданных на каждый поезд из СПб в Москву;
е) средняя цена места в купейном вагоне;
ж) поезда из СПб, делающие остановку в Окуловке и не делающие в Бологом;
з) поезда, у которых соотношение цены СВ и продолжительности в пути максимально;
и) поезда из СПб в Москву, делающие все остановки.

19. Создайте базу данных для хранения следующих сведений: оператор сотовой связи, абонент, договор, услуги, тарифы, опции, начисления, платежи. Составьте запросы, позволяющие выбрать:
    а) абонентов, пользующихся опцией АОН по какому-либо тарифу;
    б) абонентов, поменявших в прошлом году тариф «нормальный» на тариф «оптимальный»;
    в) тарифы, у которых есть одинаковые опции;
    г) количество абонентов, пользующихся каждым из тарифов;
    д) операторов, у которых средняя цена минуты выше, чем у других;
    е) операторов, доходность которых выше, чем у АБВ;
    ж) абонентов, не осуществлявших платежей в прошлом квартале;
    з) тарифы, включающие все возможные опции;
    и) абонентов, которые всегда изменяли тарифы одновременно (в один день).
20. Создайте базу данных для хранения следующих сведений: Интернет-провайдер, подключенные дома, характеристика канала, абонент, договор,

тариф, начисления, оборудование, обращение в техподдержку, результат. Составьте запросы, позволяющие выбрать:

а) абонентов, обращавшихся в техподдержку по вопросу неисправности CM440;
б) абонентов, обращавшихся в техподдержку дважды в прошлом месяце;
в) провайдеров, которые предоставляют доступ по тарифам как на скорости 50, так 70 МБит;
г) провайдеров, предлагающих более семи моделей кабельных модемов;
д) провайдеров, предлагающих наибольшее число тарифов;
е) количество абонентов для каждого провайдера по каждому из тарифов;
ж) абонентов, никогда не изменявших тариф;
з) абонентов, пользовавшихся всеми тарифами;
и) абонентов, не обращавшихся в техподдержку более трех раз в год.

21. Создайте базу данных для хранения следующих сведений: сеть ресторанов, ресторан, меню, состав блюд, бронирование столиков, чеки клиентов. Составьте запросы, позволяющие выбрать:

а) блюда, в состав которых входит говядина;
б) блюда, в состав которых входят одинаковые ингредиенты;
в) рестораны сети, в которых одинаковые блюда имеют различную цену;
г) количество блюд, предлагаемых в сети АБВ;
д) количество блюд, в которые входит каждый из ингредиентов;
е) рестораны, предлагающие в точности столько же блюд, что и заданный;
ж) рестораны, в которых нет чека, размер которого превышает 20000;
з) рестораны, в которых средний размер чека минимален;

и) рестораны, выручка которых год от года возрастает.

22. Создайте базу данных для хранения следующих сведений: почтовое отделение, обслуживаемые адреса, письмо, заказное письмо, бандероль, отправитель, получатель, прием/выдача корреспонденции. Составьте запросы, позволяющие выбрать:

а) людей, отправлявших заказные письма в прошлом месяце;

б) людей, отправлявших письма в прошлом месяце дважды по одному и тому же адресу;

в) людей, отправлявших письма в прошлом месяце и получавших ответы;

г) количество писем, пересланных из СПб в Москву в прошлом году;

д) количество корреспонденции каждого из видов между СПб и Москвой;

е) средний вес бандеролей из Москвы в СПб;

ж) почтовые отделения, количество корреспонденции в которых больше, чем в других;

з) людей, отправляющих письма всегда из одного и того же отделения;

и) людей, отправивших в прошлом году все виды корреспонденции.